Machine Learning for Time Series

Lecture 4: Data Enhancement and Preprocessings

Laurent Oudre
laurent.oudre@ens-paris-saclay.fr

Master MVA
2026-2027

Contents

1. Problem statement

2. Denoising

2.1 Filtering

2.2 Sparse approximations

2.3 Low-rank approximations

2.4 Other techniques

3. Detrending

3.1 Least-squares regression

3.2 Other approaches

4. Interpolation of missing samples

4.1 Polynomial interpolation

4.2 Low-rank interpolation

4.3 Model-based interpolation

5. Outlier removal

5.1 Isolated samples

5.2 Contiguous samples

6. Conclusion

Contents

1. Problem statement

2. Denoising

3. Detrending

4. Interpolation of missing samples

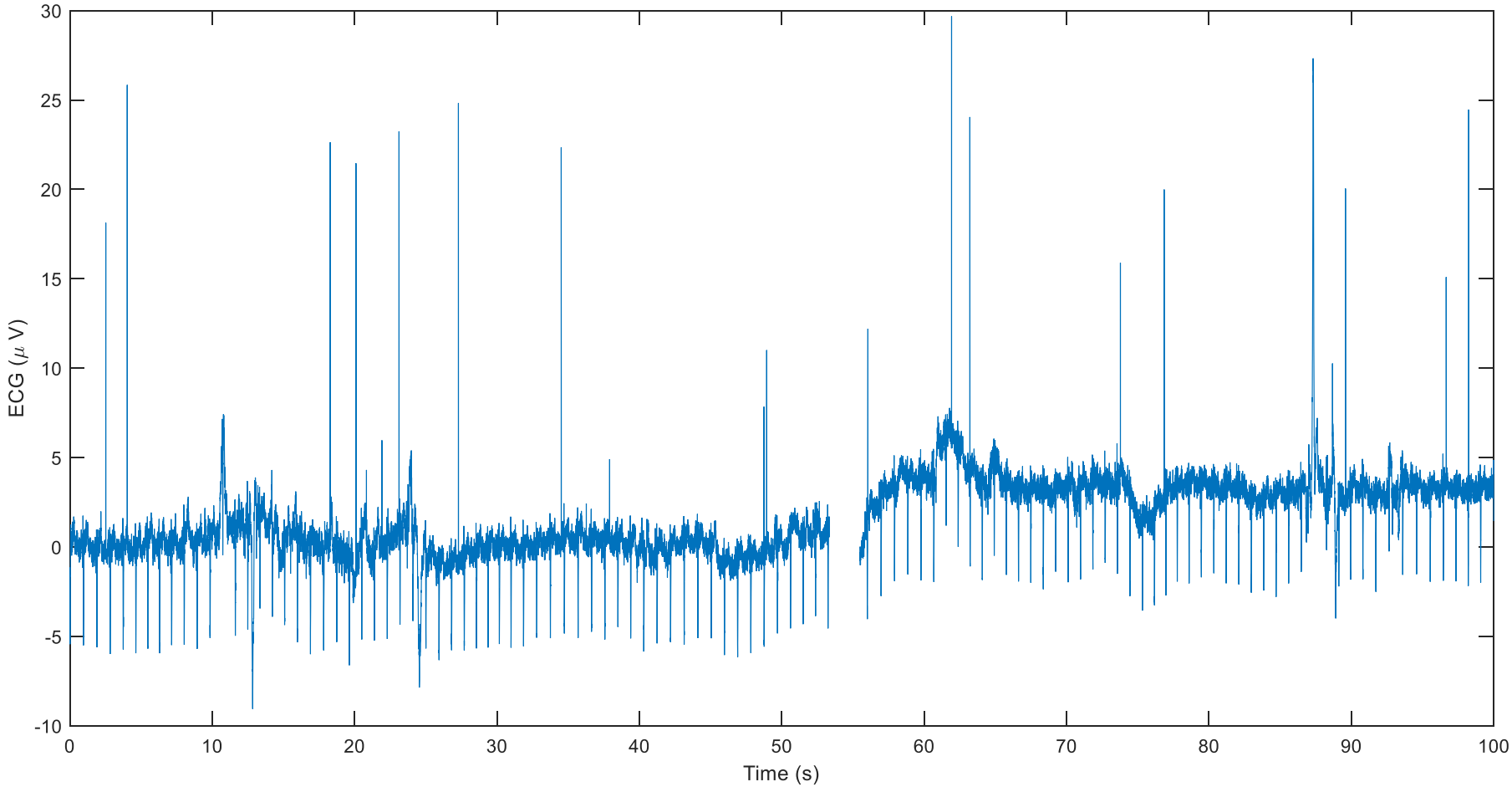
5. Outlier removal

6. Conclusion

The need for preprocessing

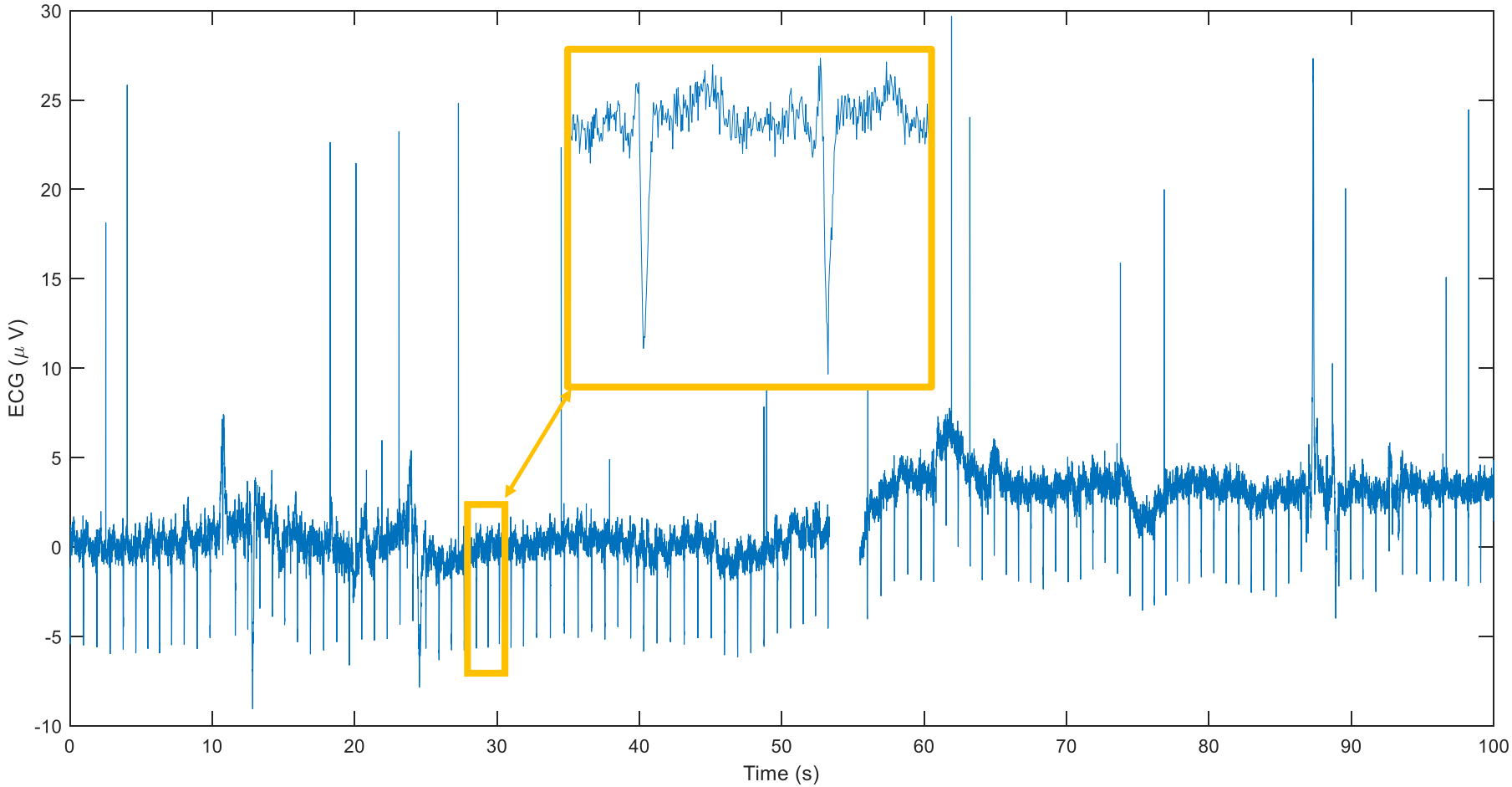
- **Typical usecase**: noisy time series with outliers and missing values
- In order to apply ML algorithms, the data scientist **needs to clean and consolidate the data**
- Time-consuming and tedious task: fortunately, ML also provides tools to that aim!
- **Careful!** All these preprocessings have a **strong impact** on the expected results and on the future learned rules!

Introductory example



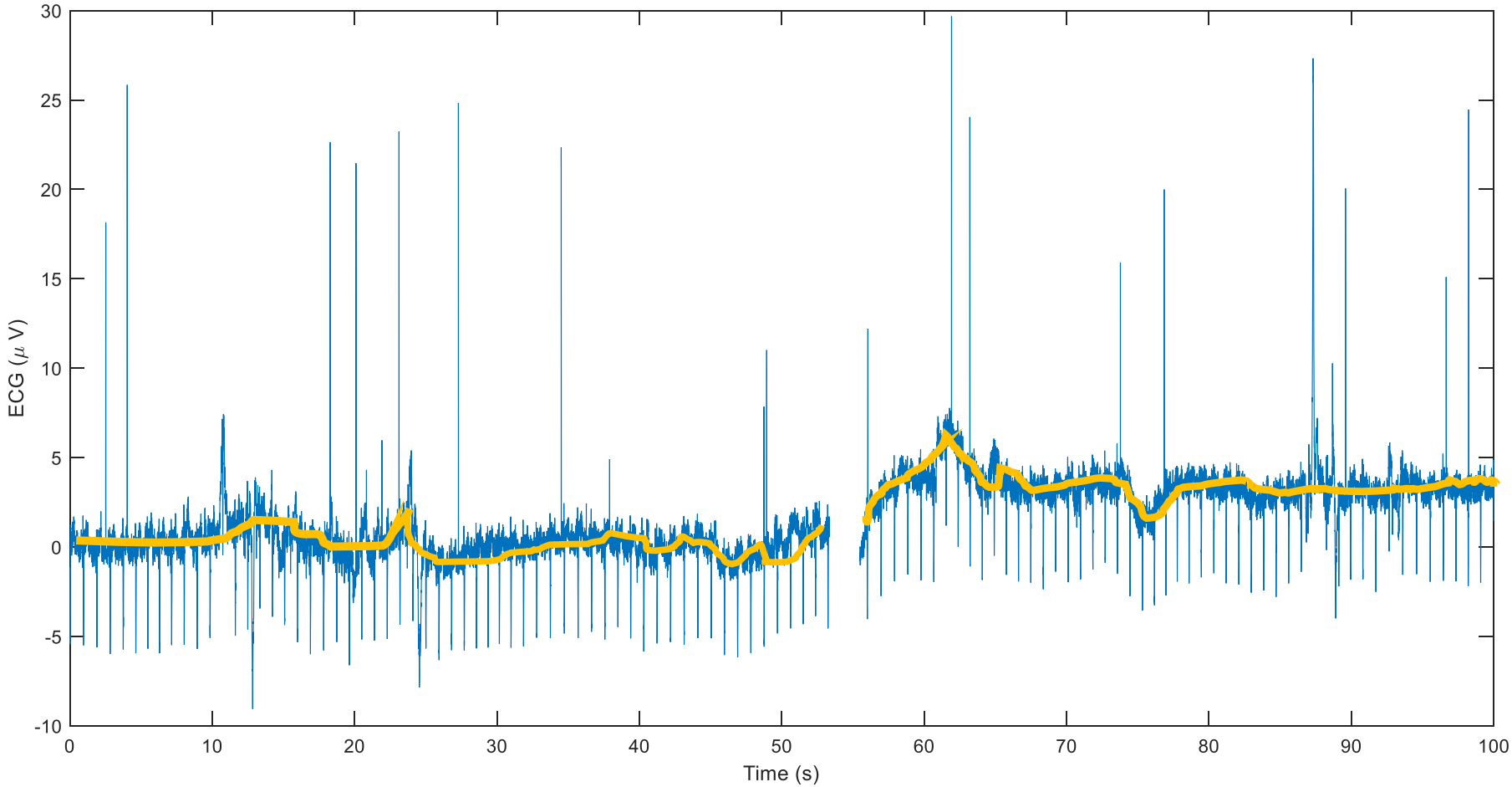
ECG signal during general anesthesia

Introductory example



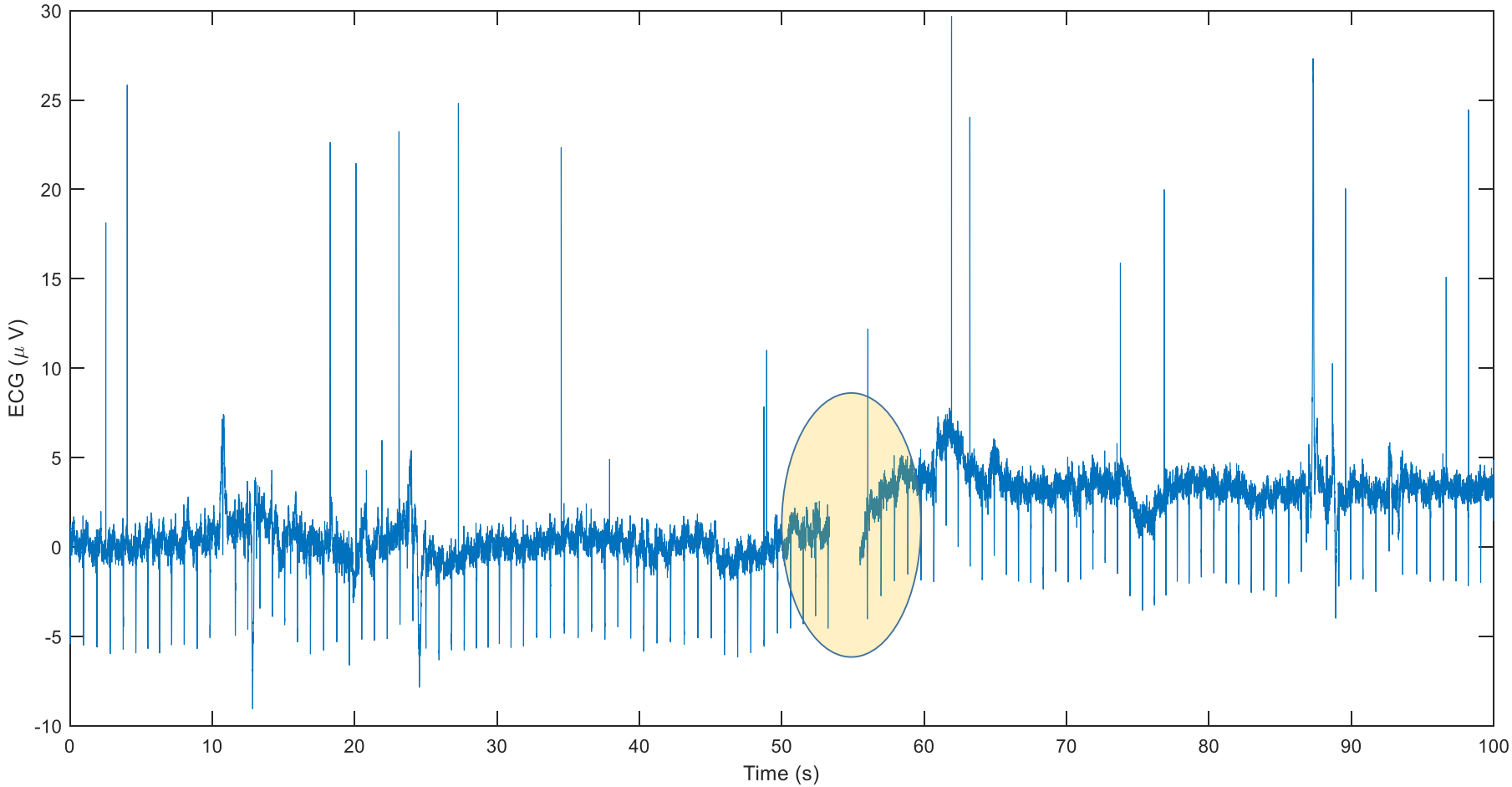
Presence of measurement noise $\rightarrow$ **Denoising**

Introductory example



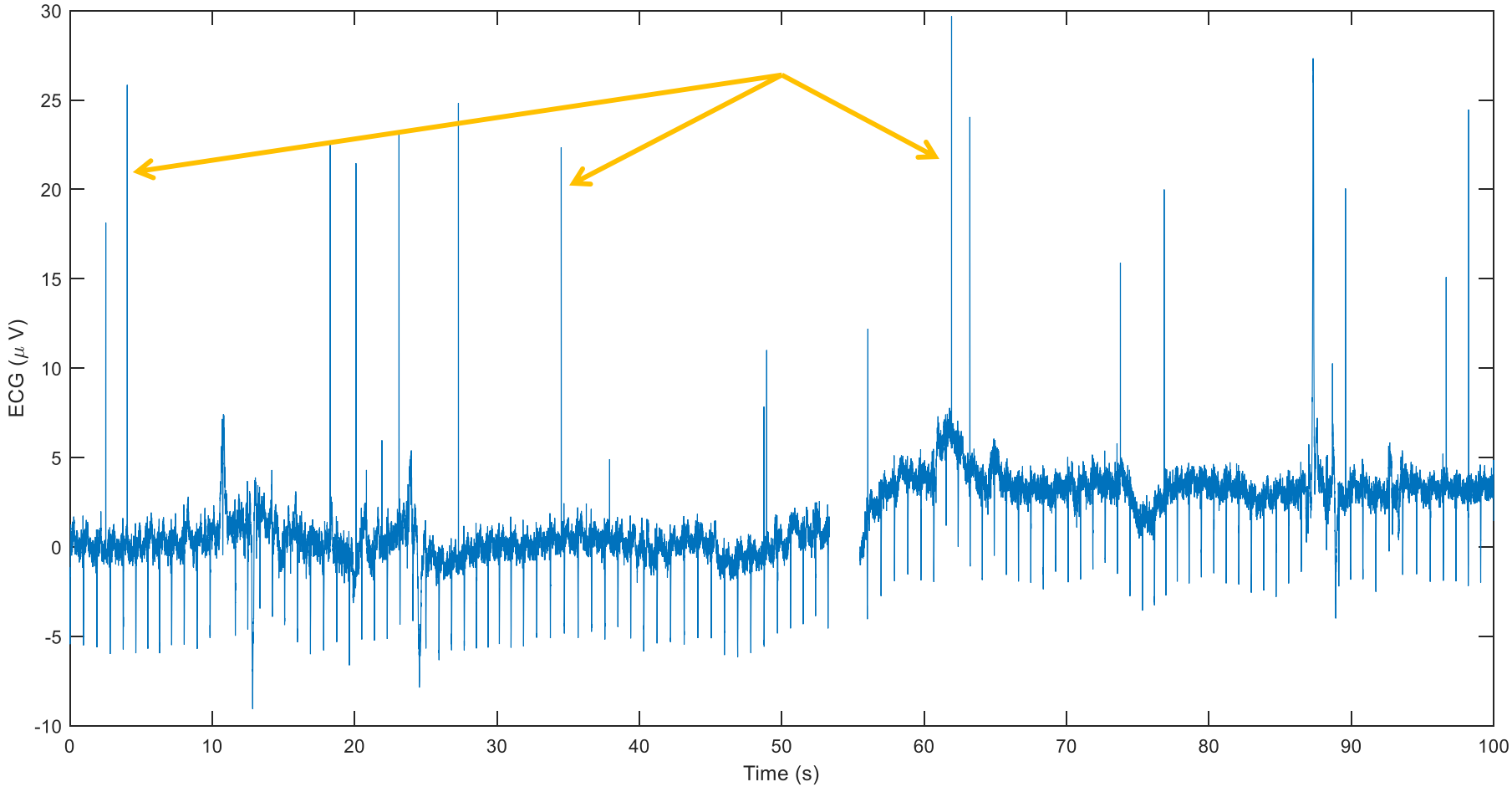
Presence of a trend $\rightarrow$ **Detrending**

Introductory example



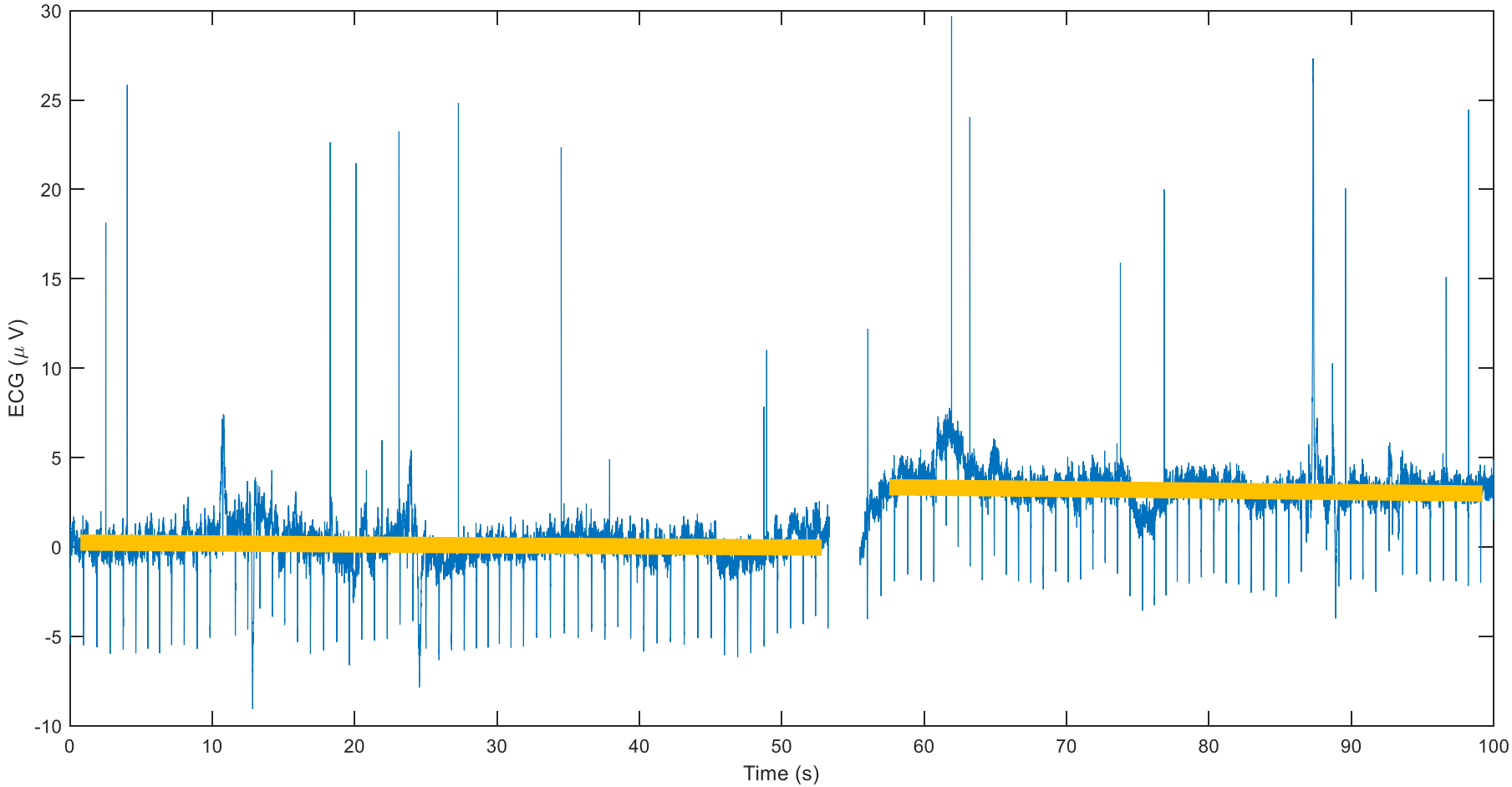
Data loss causing missing samples $\rightarrow$ **Interpolation**

Introductory example



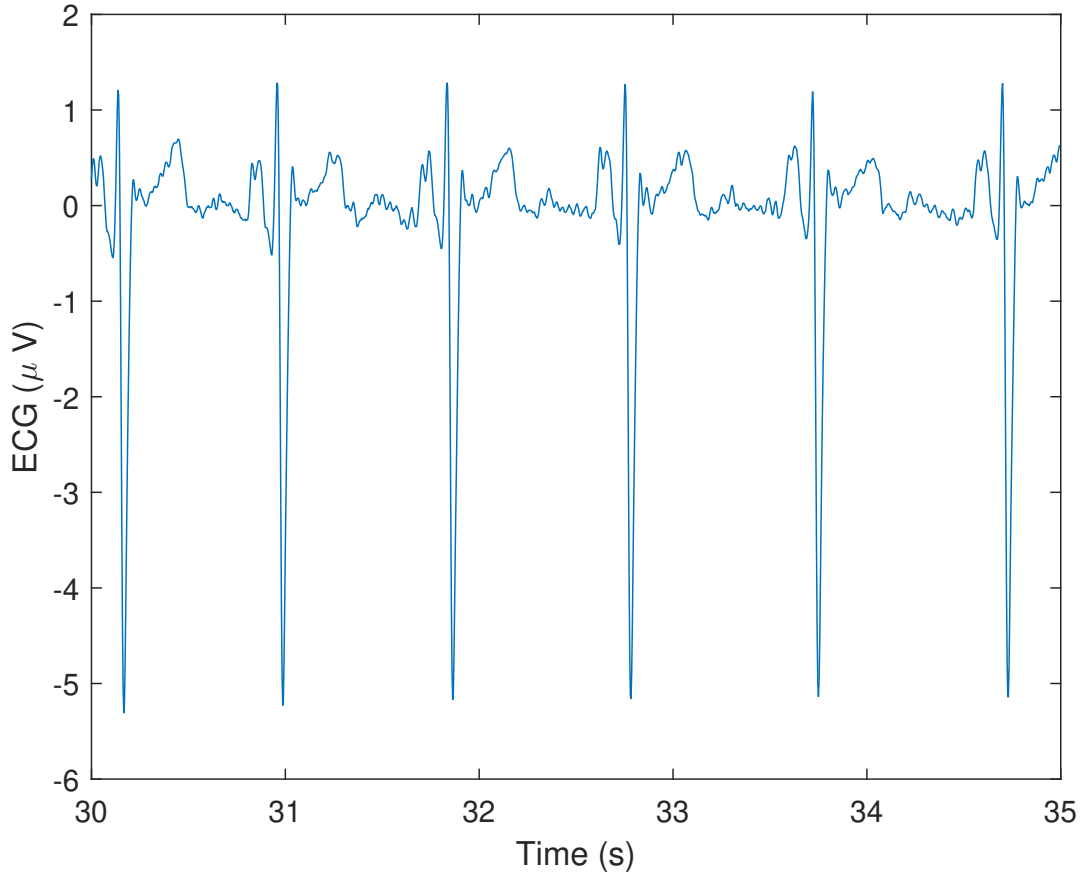
Presence of outliers $\rightarrow$ **Outlier removal and suppression of impulsive noise**

Introductory example



Break in stationarity $\rightarrow$ **Change-point detection** (see Lecture 5)

Introductory example



When all preprocessings have been performed, it becomes possible to retrieve the heartbeats and thus to perform ML

Contents

1. Problem statement

2. Denoising
2.1 Filtering
2.2 Sparse approximations
2.3 Low-rank approximations
2.4 Other techniques

3. Detrending

4. Interpolation of missing samples

5. Outlier removal

6. Conclusion

Additive white Gaussian noise (AWGN) model

The most common model for noisy signals is

$$y[n] = x[n] + b[n]$$

- $x[n]$ is the clean (unknown) signal
- $b[n]$ is the measurement noise, assumed to be additive, white and Gaussian (AWGN)
- $y[n]$ is the measured signal
- $x[n]$ and $b[n]$ are uncorrelated

Denoising

Given a noisy signal $y[n]$ corrupted by AWGN, retrieve the clean signal $x[n]$

Notion of AWGN

An AWGN $b[n]$ is:

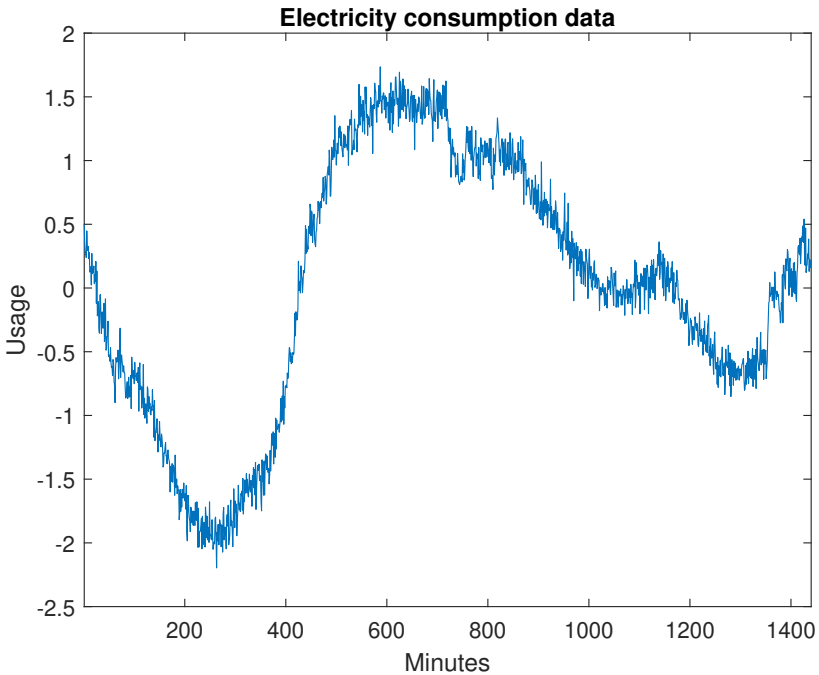
- **Additive**: the noise therefore corrupts all the samples
- **White**: stationary process with zero-mean and all samples are pairwise uncorrelated

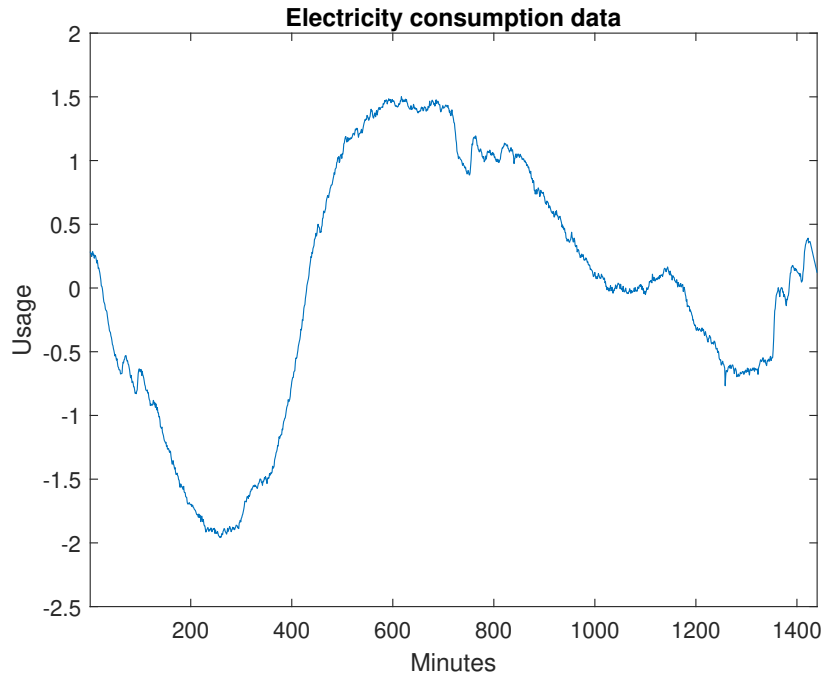
$$\gamma_b[m] = \begin{cases} \sigma^2 & m = 0 \\ 0 & \text{otherwise} \end{cases}$$

- **Gaussian**: all samples are i.i.d. according to

$$b[n] \sim \mathcal{N}(0, \sigma^2)$$

Example





How can we remove the noise component?

Filtering

- The first solution consists of using results from signal processing and statistics
- Knowing that $\gamma_x[m] = \mathbb{E}[x[n]x[n+m]]$ and using the fact that $x[n]$ and $b[n]$ are uncorrelated, we get that

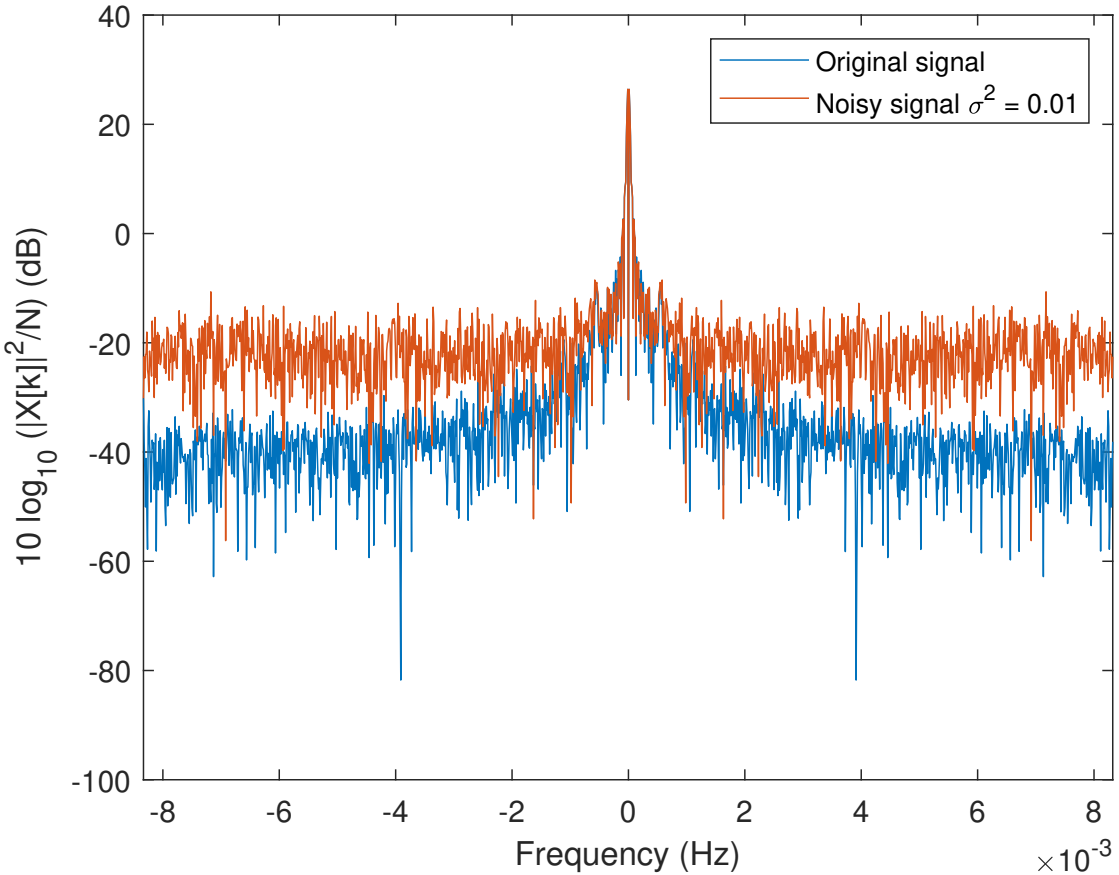
$$\gamma_y[m] = \gamma_x[m] + \gamma_b[m]$$

- By computing the DFT of this equation, we have

$$\mathbb{E}\left[|Y[k]|^2\right] = \mathbb{E}\left[|X[k]|^2\right] + N\sigma^2$$

- Adding AWGN is equivalent to **adding a constant** on the PSD of the signal (in linear scale). In the following we will use the periodogram estimator (see Lecture 2)

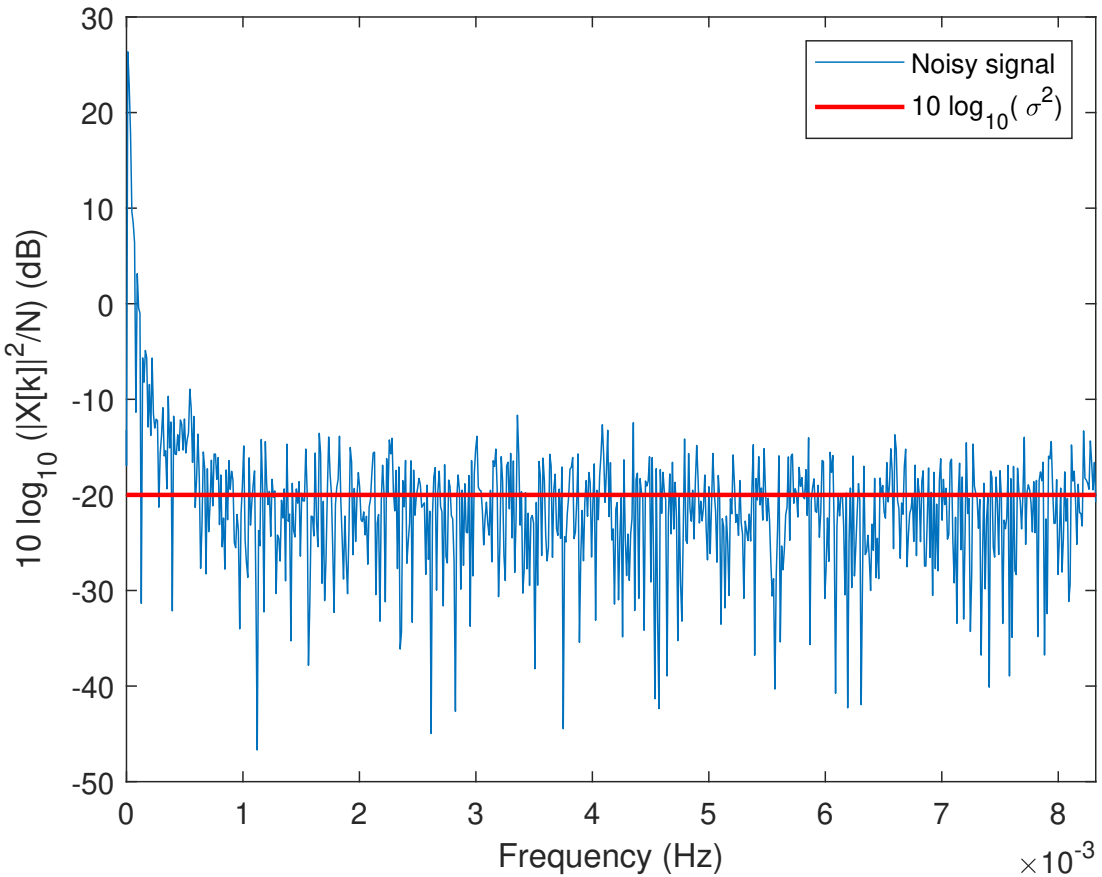
Example



In the frequency band where only AWGN is present (here with $\sigma^2 = 0.01$), the log-spectrum is equal to

$$10 \log_{10} \left(\frac{|Y[k]|^2}{N} \right) = 10 \log_{10} \left(\frac{|X[k]|^2}{N} + \sigma^2 \right) = 10 \log_{10}(0.01) = -20 \text{ dB}$$

Example



By plotting the log-spectrum of the noisy signal and knowing the noise variance σ^2, one can guess that all frequencies greater than e.g. 0.001 Hz are likely to only contain noise.

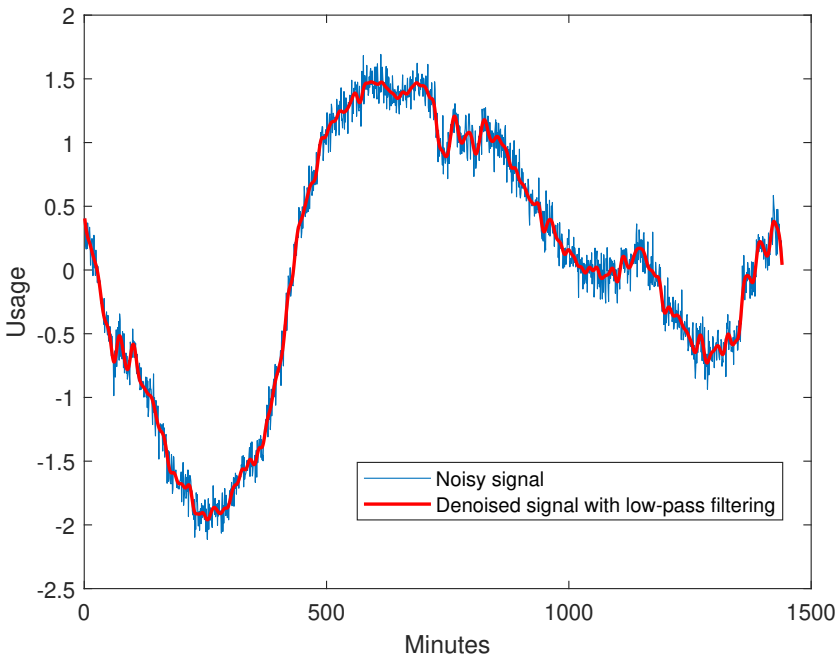
Filter design

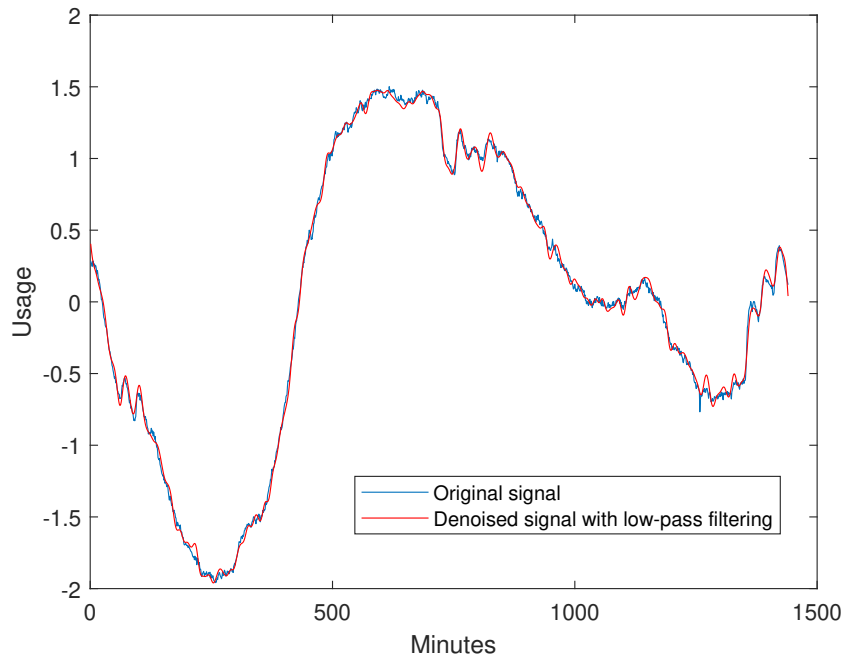
- By observing the log-spectrum of the noisy signal and using either prior knowledge on the original signal bandwidth or on the noise level, we can determine the **type of filter** and **associated cut-off frequencies** that can be used for denoising
- From that, it is only **digital filter design** (out of scope for this course !). Two popular solutions
  - **Moving average filter of length** L:

$$\hat{x}[n] = \frac{1}{L} \sum_{k=0}^{L-1} y[n-k]$$

    Low-pass filter with cut-off frequency $f_c \approx \frac{0.442947 \times F_s}{\sqrt{L^2-1}}$
  - **Butterworth filters**: can be low-pass, bandpass, etc...

Example





Low-pass filtering (Butterworth filter of order 4) with $f_c = 0.001$ Hz

Filtering vs. sparsity

- As such, filtering a signal consists of picking the frequencies that we want to keep
- Instead of designing a filter, we can attempt to retrieve a sparse frequency representation for the signal, which is equivalent to **remove the small values** on the spectrum
- **Assumption**:

$$\text{Large Fourier coefficients} \rightarrow \text{Signal}$$

$$\text{Small Fourier coefficients} \rightarrow \text{Noise}$$

- **Principle of data compression**: thresholding of small values in an appropriate representation space

$$\hat{\mathbf{x}} = \sum_{k \in \mathcal{K}} z_k \mathbf{d}_k, \quad \text{with } |\mathcal{K}| < N$$

Dictionaries

Several dictionaries can be used for denoising [Rubinstein et al., 2010]

- **Fourier dictionary** (in practice the Discrete Cosine Transform for real signals):

$$d_k[n] = \begin{cases} \frac{1}{\sqrt{N}} & k = 0 \\ \sqrt{\frac{2}{N}} \cos\left(\frac{\pi}{N}\left(n + \frac{1}{2}\right) k\right) & 1 \leq k \leq N-1 \end{cases}$$

- **Wavelet dictionary** with wavelet function $\psi(t)$ and scaling function $\phi(t)$ [Percival et al., 2000 ; Mallat, 1999]

$$\phi_{j,l}[n] = 2^{-j/2}\phi(2^{-j}n - l) \qquad \psi_{j,l}[n] = 2^{-j/2}\psi(2^{-j}n - l)$$

  The dictionary is often computed up to level j_{max} :
  - $N \times 2^{-j}$ wavelet functions $\psi_{j,l}$ at level $1 \leq j \leq j_{max}$ with l multiple of 2^j: details
  - $N \times 2^{-j_{max}}$ scaling functions $\phi_{j_{max},l}$ with l multiple of $2^{j_{max}}$: approximation
- Gabor dictionary, Modified Discrete Cosine Transform (MDCT)...

Sparse coding

Given an input dictionary $\mathbf{D}$, the denoising task is equivalent to a **sparse coding** task, and all previously seen algorithms can be used to that aim (see Lecture 3)

- ℓ_0-based algorithms with **hard thresholding**

$$\mathbf{z}^* = \underset{\substack{\mathbf{z} \\ \|\mathbf{z}\|_0 = K_0}}{\operatorname{argmin}} \|\mathbf{y} - \mathbf{D}\mathbf{z}\|_2^2$$

  Only keep the K_0 largest coefficients in the decomposition

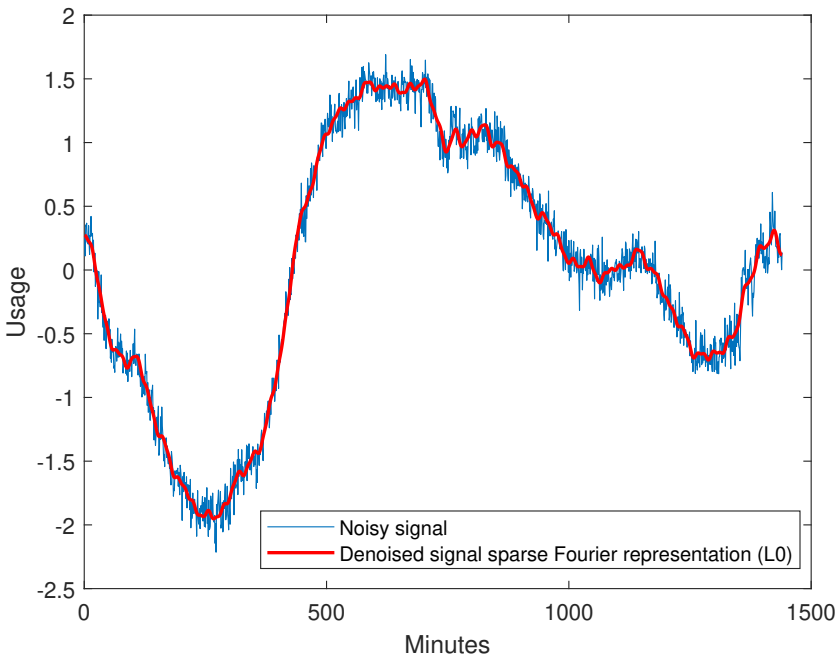
- ℓ_1-based algorithms with **soft thresholding**

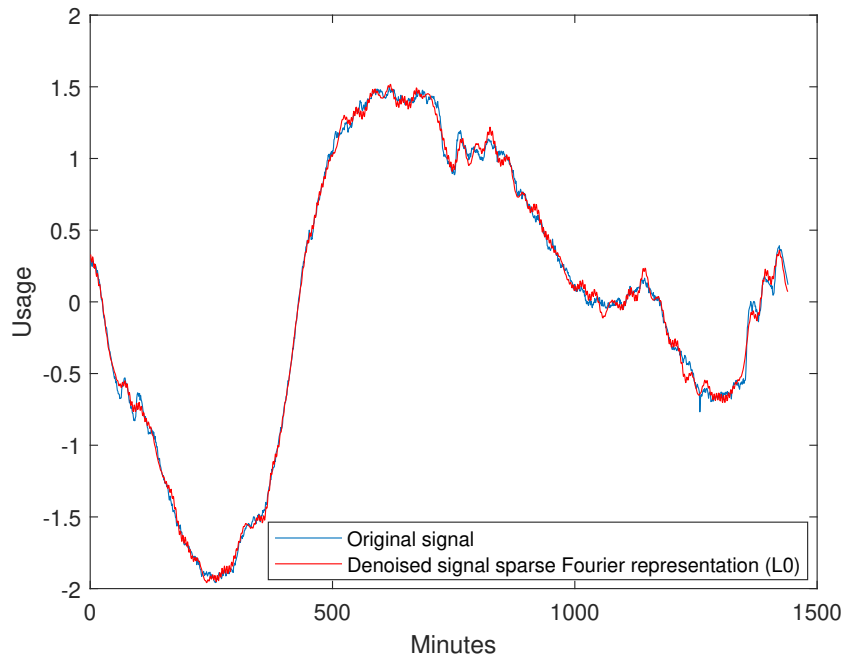
$$\mathbf{z}^* = \underset{\mathbf{z}}{\operatorname{argmin}} \|\mathbf{y} - \mathbf{D}\mathbf{z}\|_2^2 + \lambda \|\mathbf{z}\|_1$$

  Set to zero the coefficients that are lower than a given threshold (and shrink the other ones)

$$\mathcal{S}_\lambda(\mathbf{z}) = \operatorname{sign}(\mathbf{z}) \times \max(|\mathbf{z}| - \lambda, 0)$$

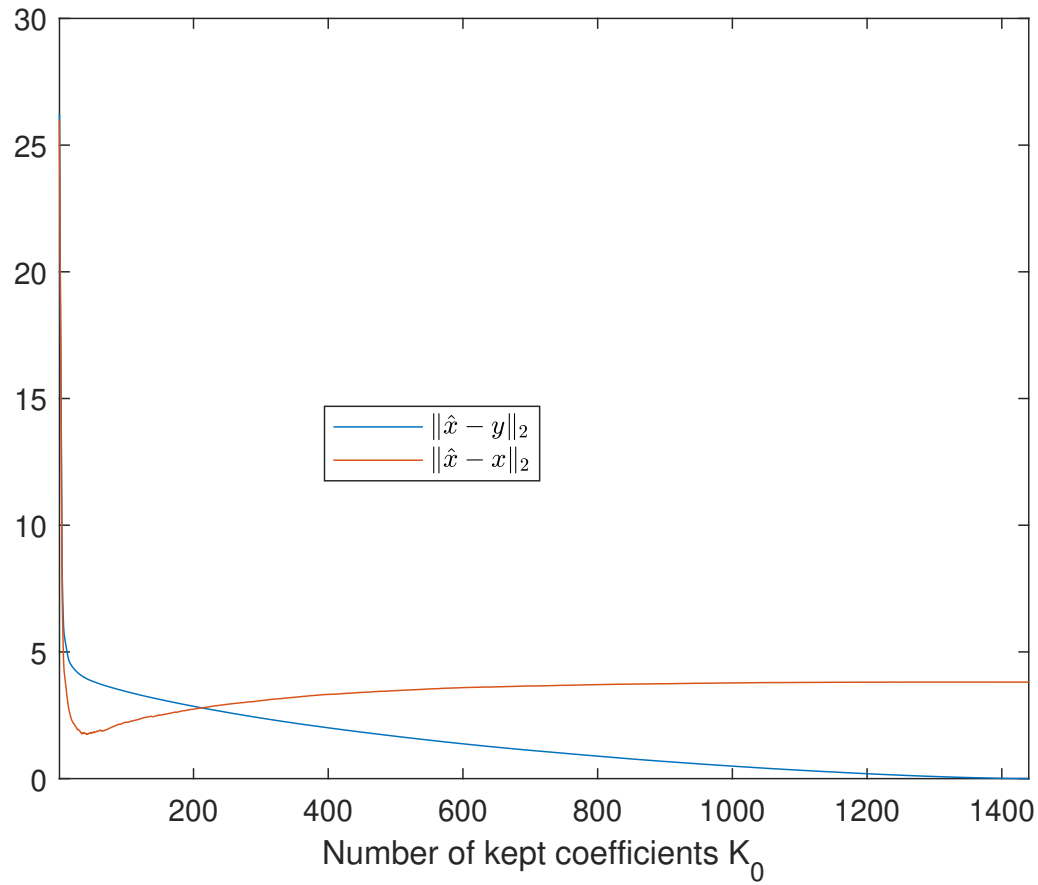
Example





Matching pursuit with Fourier dictionary and $K_0 = 40$

Example



Influence of the K_0 parameter on the denoising performances
Blue: distance to the noisy signal, red: distance to the clean signal

How to set K_0 or λ

- The parameters depend on the used dictionary (in particular on orthogonality properties of the atoms) and on the used algorithm
- **Heuristics** :
  - Use a training set
  - Use for λ a certain percentage of $\lambda_{max} = \left\| \mathbf{D}^\top \mathbf{y} \right\|_\infty$
  - Empirical observation of the distribution of activations can also be used to choose the parameters (find an elbow on the curve)
  - Stochastic strategies: divide the over redundant dictionary into several small dictionaries and average the decomposition made on these dictionaries
- **Statistics** (often require a probabilistic model for the data and/or noise) :
  - Some statistical results on estimators can be used to have an idea of the range of relevant parameters (Stein's Unbiased Risk Estimate (SURE), Minimax criteria) : see next slides for an example
  - Model selection strategies can also be used (see Lecture 5 and Tutorial session 3)

How to determine the stopping criterion?

- Example for the **Matching Pursuit** algorithm. For greedy denoising approaches (such as Matching Pursuit), a good denoising strategy would be to stop once the atoms in the dictionary have captured all relevant information on the signal, i.e. when the residual is composed of **pure noise**
- An interesting measure is the **normalized coherence** between the signal $\mathbf{y}$ and the dictionary $\mathbf{D}$

$$\rho_{\mathbf{D}}(\mathbf{y}) = \max_{\mathbf{d} \in \mathbf{D}} \frac{|\langle \mathbf{y}, \mathbf{d} \rangle|}{\|\mathbf{y}\|_2}$$

Stopping criteria for matching pursuit

- By denoting $\mathbf{r}^{(\ell)}$ the residual at iteration ℓ,

$$\mathbf{r}^{(\ell)} = \mathbf{r}^{(\ell-1)} - \langle \mathbf{r}^{(\ell-1)}, \mathbf{d}^* \rangle \mathbf{d}^*$$

  where $\mathbf{d}^*$ is the atom most correlated to $\mathbf{r}^{(\ell-1)}$
- Basic calculations give that

$$\frac{\|\mathbf{r}^{(\ell)}\|_2^2}{\|\mathbf{r}^{(\ell-1)}\|_2^2} = 1 - \rho_{\mathbf{D}}^2(\mathbf{r}^{(\ell-1)})$$

- The decreasing of the L2 norm of the residual is therefore linked to the normalized coherence of the residual with the dictionary
  - If $\rho_{\mathbf{D}}(\mathbf{r}^{(\ell-1)})$ is large, it is worth continuing
  - If $\rho_{\mathbf{D}}(\mathbf{r}^{(\ell-1)})$ becomes too small, the algorithm can stop
- When can we say that the coherence becomes **too low**?

Stopping criteria for matching pursuit

- One interesting question is therefore: what is the value of $\rho_{\mathbf{D}}(\mathbf{r})$ when the residual $\mathbf{r}$ is pure noise ?
- If $\mathbf{r}$ is pure random with a known distribution $p(\mathbf{r})$ (e.g. AWGN), we can be interested in the quantity

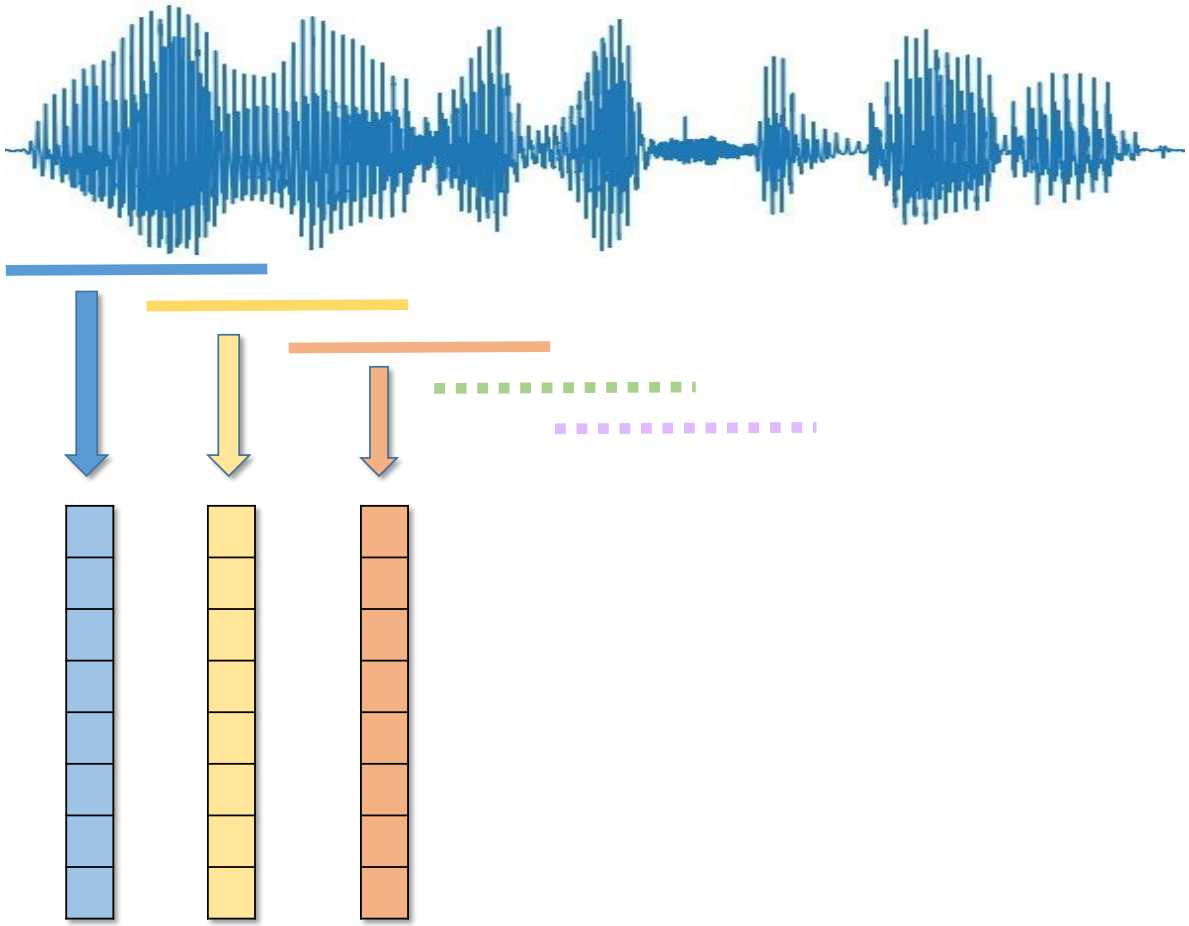
$$\rho_{p(\mathbf{r})}(\mathbf{D}) = \mathbb{E}_{p(\mathbf{r})}\left[\rho_{\mathbf{D}}(\mathbf{r})\right]$$

- Intuitively, denoising for $\mathbf{x}$ can then be achieved by stopping when the normalized coherence of the residual has the same order of magnitude as this value
- How to compute $\rho_{p(\mathbf{r})}(\mathbf{D})$?
  - Use a training set of noise signals
  - Use statistical considerations with parametrized distribution (see mini-project)

Use of adaptive dictionaries

- Instead of using off-the-shelf dictionaries, we can learn the representation directly from the signal: **dictionary learning** (see Lecture 3)
- Use of the **trajectory matrix X**: matrix representation of the input signal frames
- Noise is random: when sparsity is enforced, the approximation tends to **only model signal**

Trajectory matrix



N_w: window length, N_o: overlap length

N_w rows, $N_f = \lfloor \frac{N-N_w}{N_w-N_o} \rfloor + 1$ columns

Dictionary learning

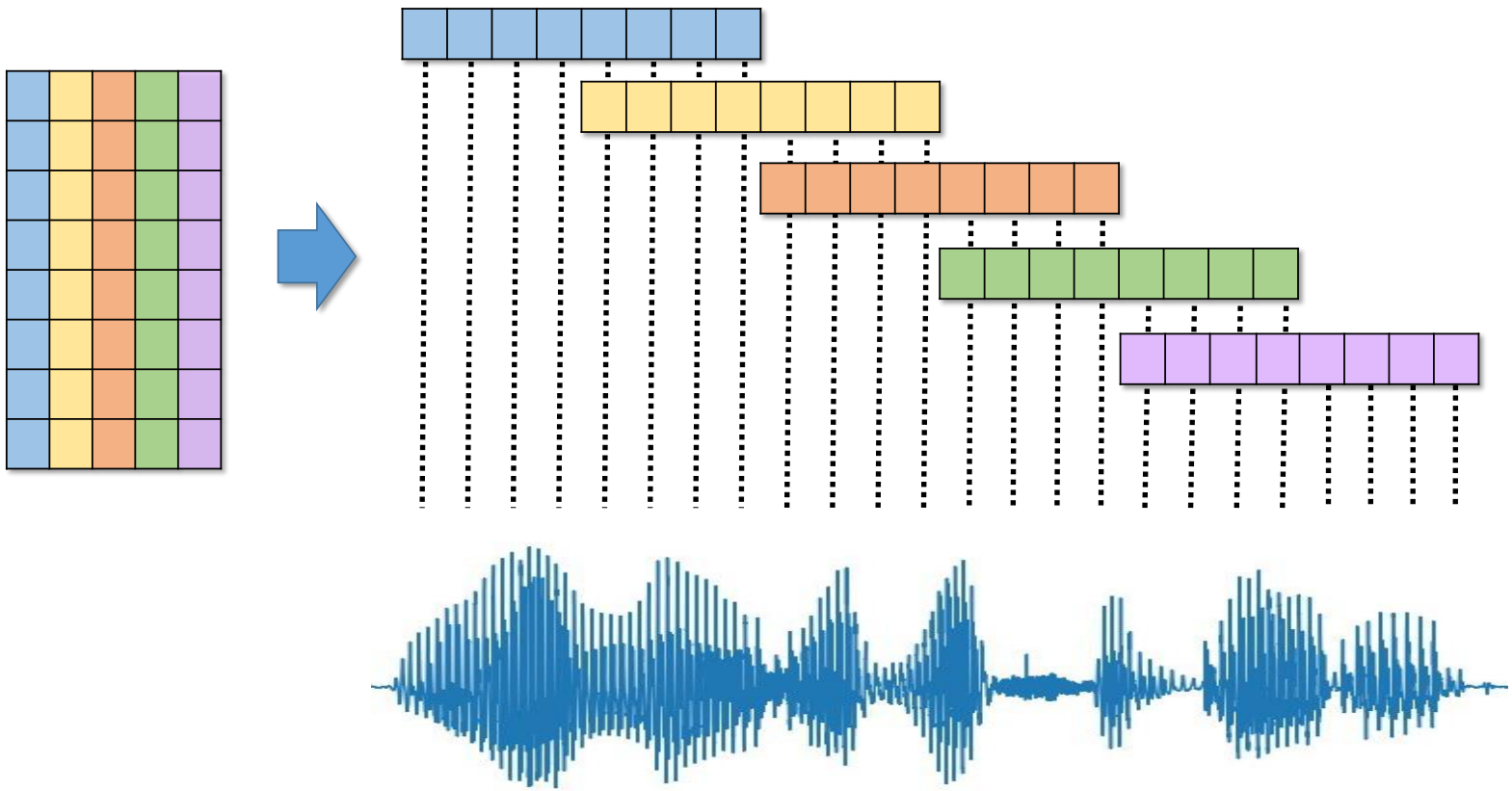
With algorithms already described in Lecture 3, compute an approximation of the trajectory matrix

$$\hat{\mathbf{X}} = \mathbf{D}\mathbf{Z} \approx \mathbf{X}$$

- $\mathbf{D} \in \mathbb{R}^{N_w \times K}$: dictionary composed of K atoms
- $\mathbf{Z} \in \mathbb{R}^{K \times N_f}$: sparse activations (sparsity level specified with K_0 or λ)

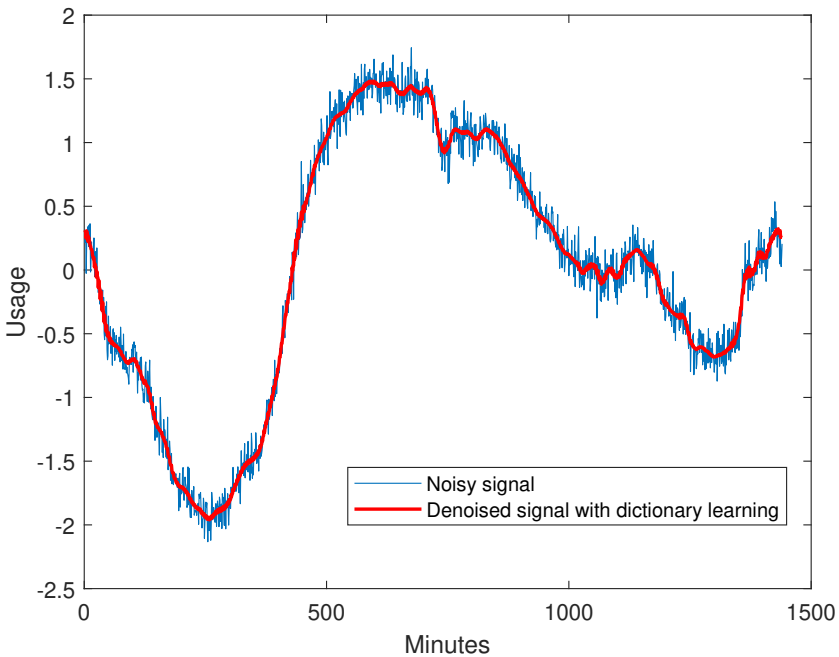
Each frame is approximated as a **sparse linear combination** of the learned atoms

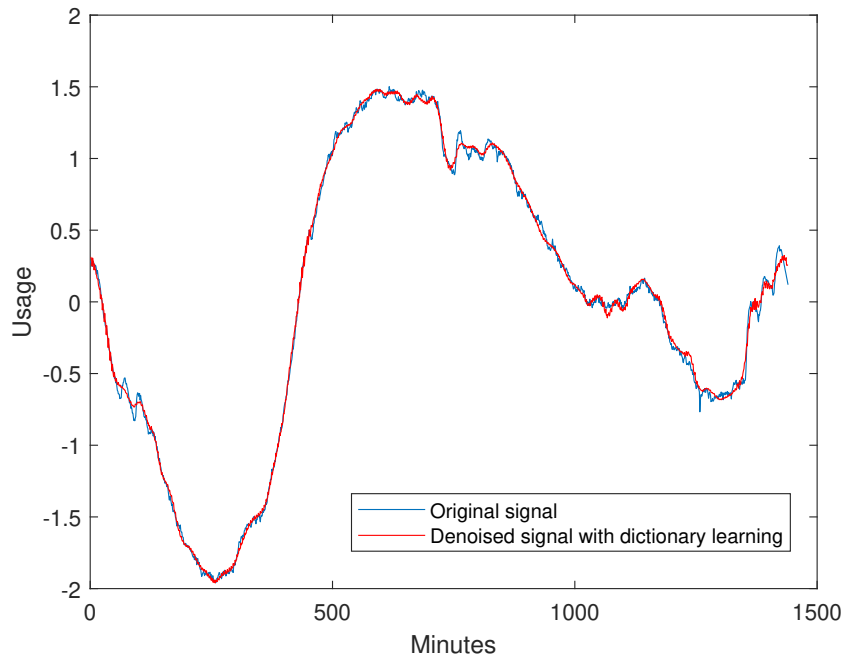
Reconstruction from the approximated trajectory matrix



Unfolding of the matrix and averaging along overlapping frames

Example





Dictionary learning with $K = 5$, $K_0 = 2$, $N_w = 32$, $N_o = 28$

Trajectory matrix

- When $N_o = N_w - 1$, we have $N_f = N - N_w + 1$ and the trajectory matrix $\mathbf{X} \in \mathbb{R}^{N_w \times N_f}$ has a particular form

$$\mathbf{X} = \begin{pmatrix} x[0] & \cdots & x[N - N_w] \\ x[1] & \cdots & x[N - N_w + 1] \\ \vdots & \ddots & \vdots \\ x[N_w - 1] & \cdots & x[N - 1] \end{pmatrix}$$

- It contains **all** N_f sequences of length N_w in the time series
- Low-rank approximations attempt to reconstruct matrix $\mathbf{X}$ as the sum of $K < \min(N_w, N_f)$ **rank-one matrices**

Singular Value Decomposition

Assuming that $N_w < N_f$, the **Singular Value Decomposition (SVD)** of matrix $\mathbf{X}$ writes:

$$\mathbf{X} = \underbrace{\mathbf{U}}_{N_w \times N_w} \underbrace{\mathbf{\Sigma}}_{N_w \times N_f} \underbrace{\mathbf{V}^\top}_{N_f \times N_f}$$

where

- $\mathbf{U}$ and $\mathbf{V}$ are orthogonal matrices
- $\mathbf{\Sigma}$ is a diagonal matrix containing on its first diagonal at most N_w singular values $\lambda_1 \geq \ldots \geq \lambda_{N_w}$

$$\mathbf{X} = \sum_{k=1}^{N_w} \lambda_k \mathbf{u}_k \mathbf{v}_k^\top$$

Interpretation of the singular values

- For a zero-mean stationary signal, the lag-covariance matrix for lag N_w can be estimated as:

$$\mathbf{C_X} = \frac{1}{N_f}\mathbf{X}\mathbf{X}^\top$$

- Positive semi-definite matrix with eigen decomposition

$$\mathbf{C_X} = \tilde{\mathbf{U}}\tilde{\mathbf{\Sigma}}\tilde{\mathbf{T}}^\top$$

  $\tilde{\lambda}_k$ corresponds to the contribution of the direction given by k^{th} eigenvector to the global variance (see Lecture 2 on **Principal Component Analysis (PCA)**)

- Basic computations give that

$$\lambda_k \propto \sqrt{\tilde{\lambda}_k}$$

  which provides a natural interpretation of the singular values of the trajectory matrix

Singular Spectrum Analysis (SSA)

This principle is the core of the **Singular Spectrum Analysis (SSA)** algorithm [Vautard et al., 1992]:

1. Compute the SVD of the trajectory matrix

$$\mathbf{X} = \sum_{k=1}^{N_w} \lambda_k \mathbf{u}_k \mathbf{v}_k^\top$$

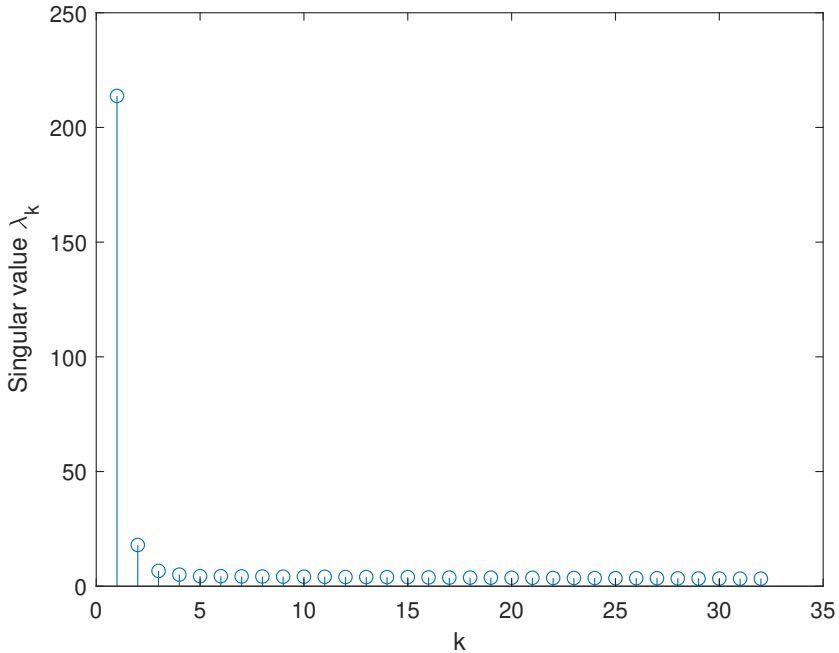
2. By analyzing the singular value distribution, form groups $\mathcal{K}_1, \mathcal{K}_2, \ldots, \mathcal{K}_M$ of singular values corresponding to similar phenomena

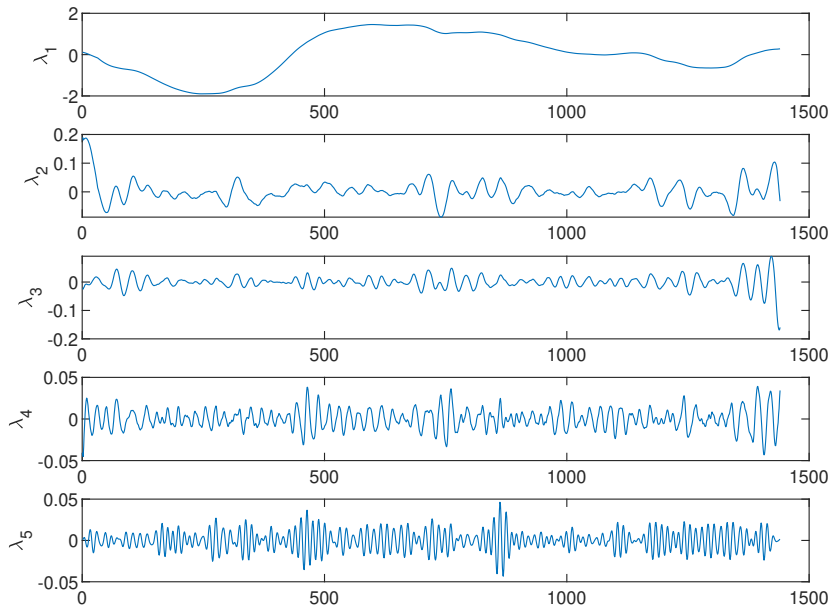
$$\mathbf{X} \approx \sum_{k \in \mathcal{K}_1} \lambda_k \mathbf{u}_k \mathbf{v}_k^t + \ldots + \sum_{k \in \mathcal{K}_M} \lambda_k \mathbf{u}_k \mathbf{v}_k^t$$

Using SSA for denoising

- Intuitively, for reasonable signal-to-noise ratio, signal should be dominant and thus corresponds to the **largest singular values**
- By plotting the singular values λ_k as a function of k, it is possible to detect and group the different phenomena within the time series
- For denoising, it is common to remove all components corresponding to small singular values
- **Choice of** N_w (only parameter): longest periodicity captured by SSA

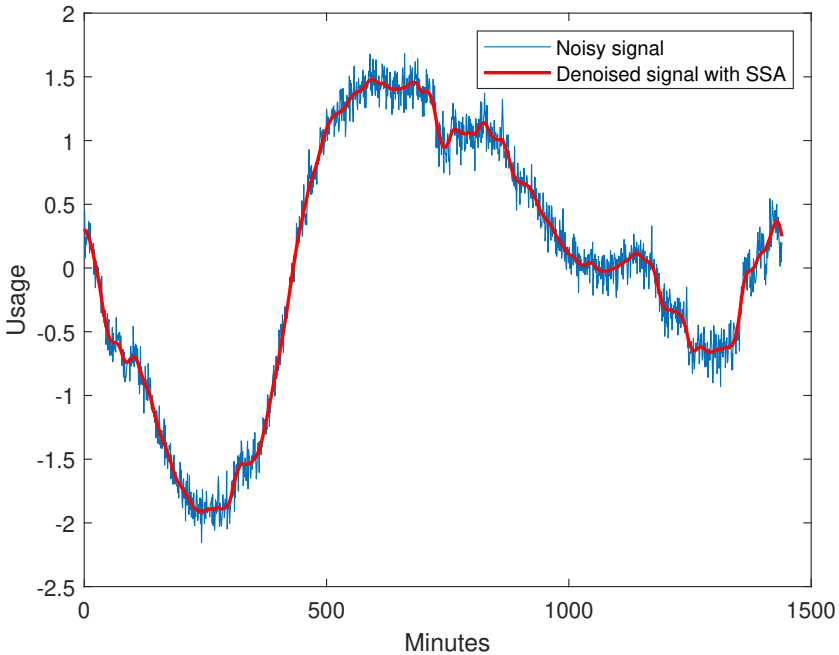
Example

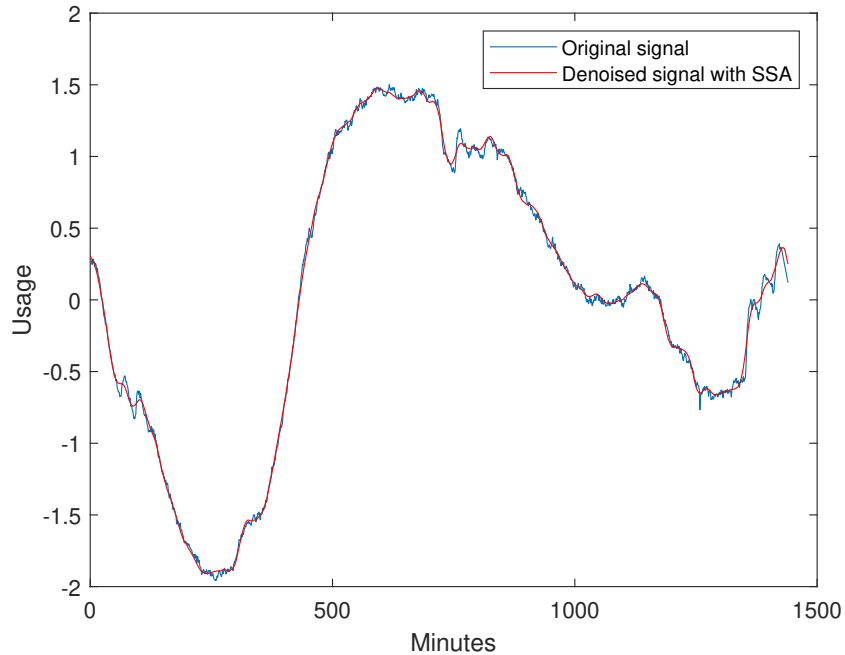




Singular values and reconstructed components with $N_w = 32$. From the graphs it appears that the two first singular values are likely to be signal

Example





Denoising with SSA with $N_w = 32$ and using only the two first components.

Other techniques

Several other decomposition techniques can be used:

- **Independent Component Analysis (ICA)**: decompose the signal into the sum of statistically independent components [Comon, 1994]
  - Useful for blind source separation and unmixing (e.g. in EEG data or audio)
  - Algorithms based on the optimization of several measures of independence (mutual information, gaussianity etc.)
- **Empirical Mode Decomposition (EMD)**: decompose the signal into the sum of oscillatory modes with various amplitude and frequency [Flandrin et al., 2004 ; Boudraa et al., 2006]
  - Useful for denoising but also detrending
  - Algorithms based on the iterative modeling of the signal as splines
- **Seasonal-Trend decomposition by Loess (STL)**: alternate a smoothing of the seasonal subseries and a smoothing of the deseasonalized signal [Cleveland et al., 1990]
  - Useful for detrending and deseasonalizing jointly
  - The seasonal pattern is allowed to change over time, and no functional form is imposed.

Contents

1. Problem statement

2. Denoising

3. Detrending
3.1 Least-squares regression
3.2 Other approaches

4. Interpolation of missing samples

5. Outlier removal

6. Conclusion

Trend+Seasonality model

The trend+seasonality model writes as

$$x[n] = \underbrace{\alpha_1\beta_1(nT_s) + \ldots + \alpha_j\beta_j(nT_s)}_{x^{trend}[n]} + \underbrace{\alpha_{j+1}\beta_{j+1}(nT_s) + \ldots + \alpha_d\beta_d(nT_s)}_{x^{seasonality}[n]} + b[n]$$

- **Seasonality**: pseudo-periodic component
- **Trend**: smooth variations, systematic increase or decrease in the data

Detrending

Given a signal $x[n]$, estimate and remove the trend component $x^{trend}[n]$

An ill-posed decomposition

- The decomposition $x[n] = x^{trend}[n] + x^{seasonality}[n] + b[n]$ is **not identifiable** : any slowly varying part of the residual can be moved into the trend, and conversely
- What separates them is a convention on what counts as slow, and this convention is a modelling choice, not a property of the data
- Unlike noise, which is defined by a statistical property, a trend is only defined by a **scale**
- Two consequences :
  - There is no universally correct detrending, only one adapted to the question asked
  - What is removed must be examined, not only what is kept

Standard models

The most common trend models are:

- **Constant trend**

$$x^{trend}[n] = \alpha_0$$

- **Linear trend**

$$x^{trend}[n] = \alpha_1 (nT_s) + \alpha_0$$

- **Polynomial trend**

$$x^{trend}[n] = \sum_{k=0}^{K} \alpha_k (nT_s)^k$$

Least-squares regression

- **Least-squares estimator**: minimization of

$$\|\mathbf{x} - \boldsymbol{\beta}\boldsymbol{\alpha}\|_2^2$$

where

$$\boldsymbol{\beta} = \begin{pmatrix} \beta_0(0) & \cdots & \beta_K(0) \\ \beta_0(T_s) & \cdots & \beta_K(T_s) \\ \vdots & \ddots & \vdots \\ \beta_0((N-1)T_s) & \cdots & \beta_K((N-1)T_s) \end{pmatrix}$$

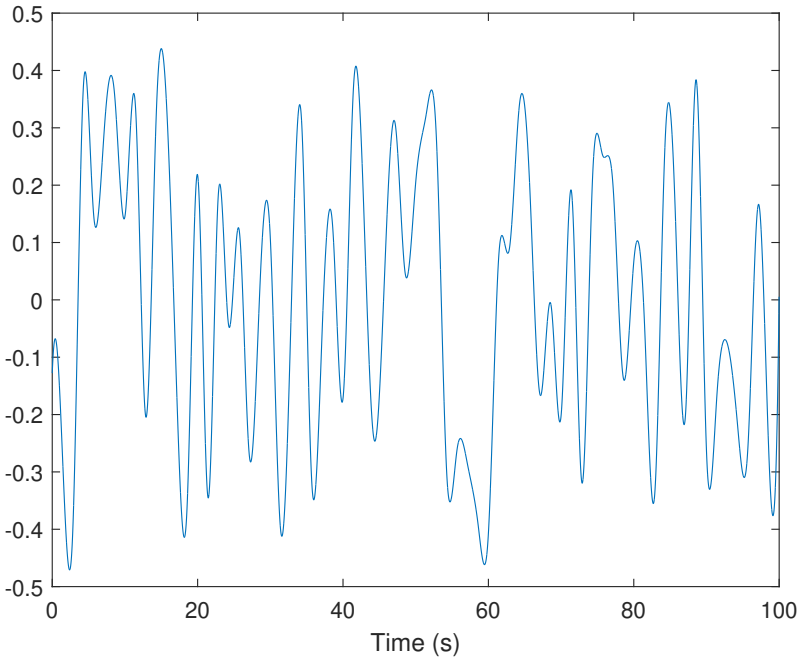
- **Closed form solution**

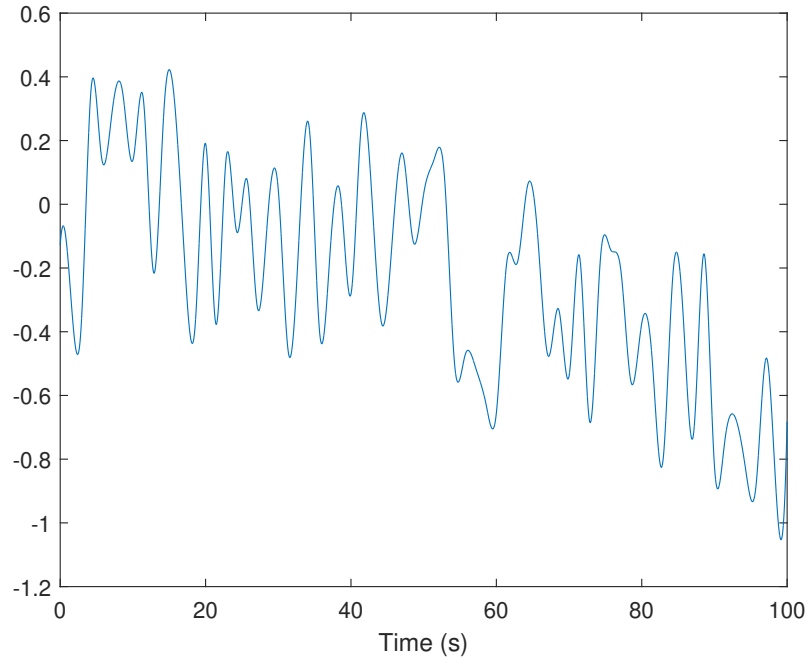
$$\hat{\boldsymbol{\alpha}} = \left(\boldsymbol{\beta}^T\boldsymbol{\beta}\right)^{-1}\boldsymbol{\beta}^T\mathbf{x}$$

- **Estimation of the trend**

$$\mathbf{x}^{\mathbf{trend}} = \boldsymbol{\beta}\hat{\boldsymbol{\alpha}}$$

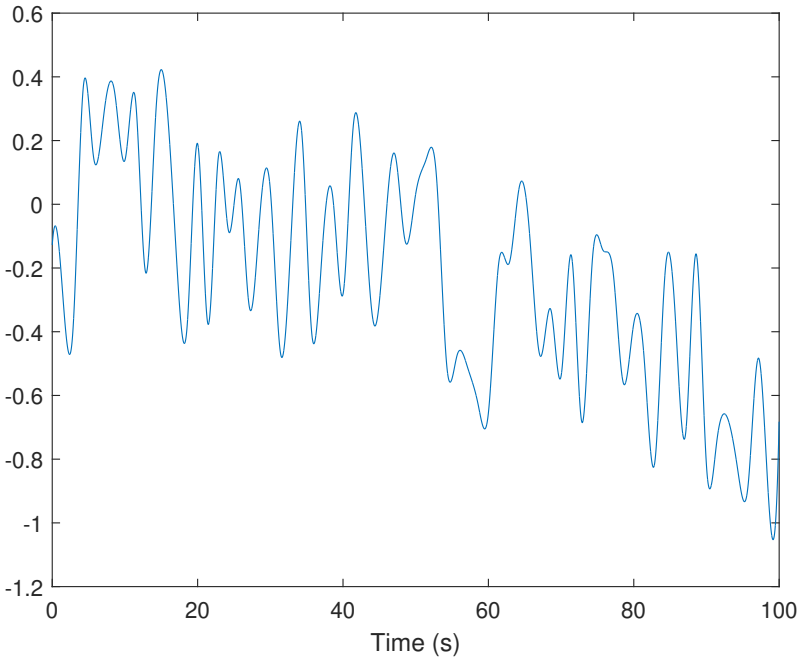
Example

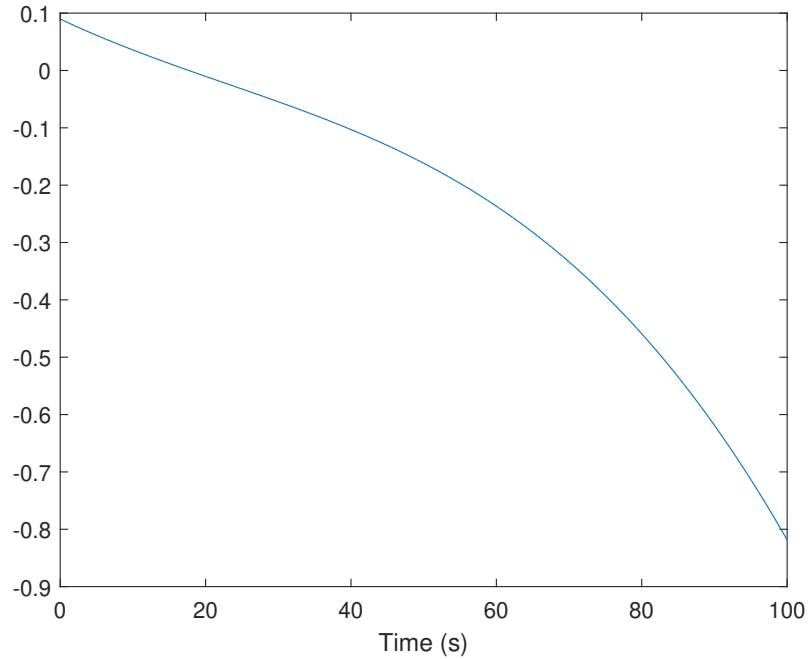




Signal with/without trend

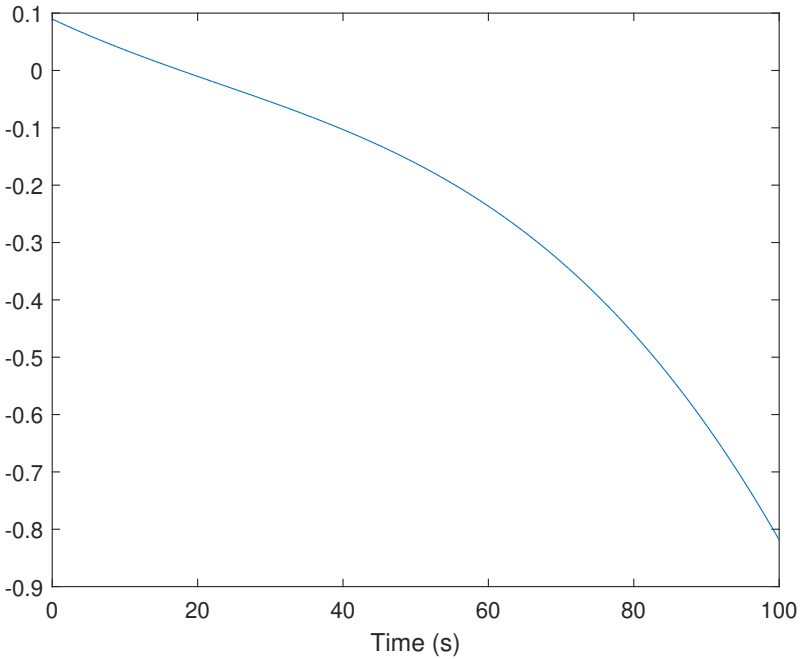
Example

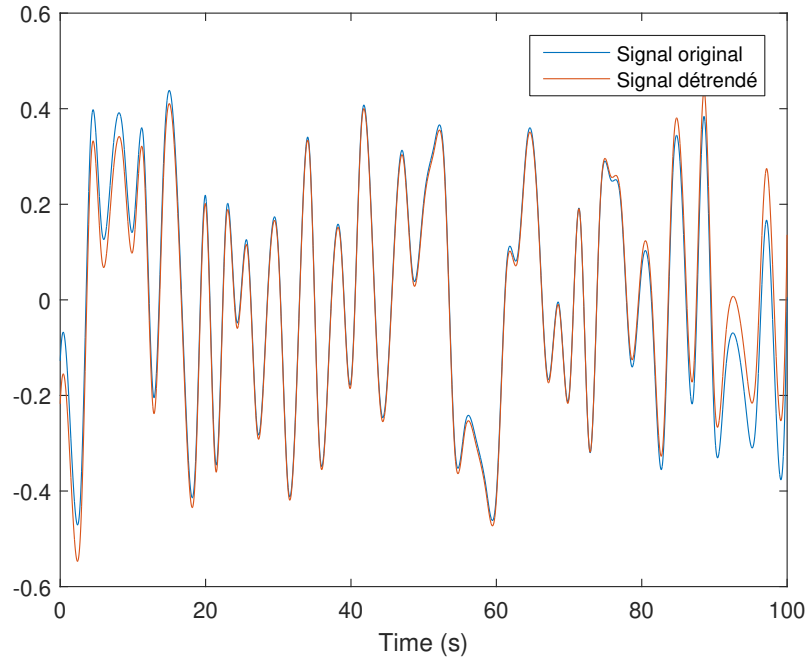




Regression on polynomials of order 3

Example





Regression on polynomials of order 3

Other approaches

Other approaches for detrending include

- **Differencing**, as a trend disappears when taking successive differences $x[n] - x[n-1]$
  - One difference removes a linear trend, two a quadratic one... (see Lecture 3)
  - No model and no parameter to estimate, but the operation **amplifies the high frequencies**, hence the noise
- **Filtering techniques**, as trends often correspond to low frequencies or smooth components (low-pass/bandpass filters, Fourier or wavelets thresholding...)
  - Note that least-square regression is itself a filter : $\mathbf{x}^{\mathbf{trend}} = \boldsymbol{\beta}\left(\boldsymbol{\beta}^T\boldsymbol{\beta}\right)^{-1}\boldsymbol{\beta}^T\mathbf{x}$ is a projection, hence a linear operation on $\mathbf{x}$
- **Decomposition techniques**, as trends may be considered independent of the seasonality and/or the noise component (EMD, SSA, ICA...)

Contents

1. Problem statement

2. Denoising

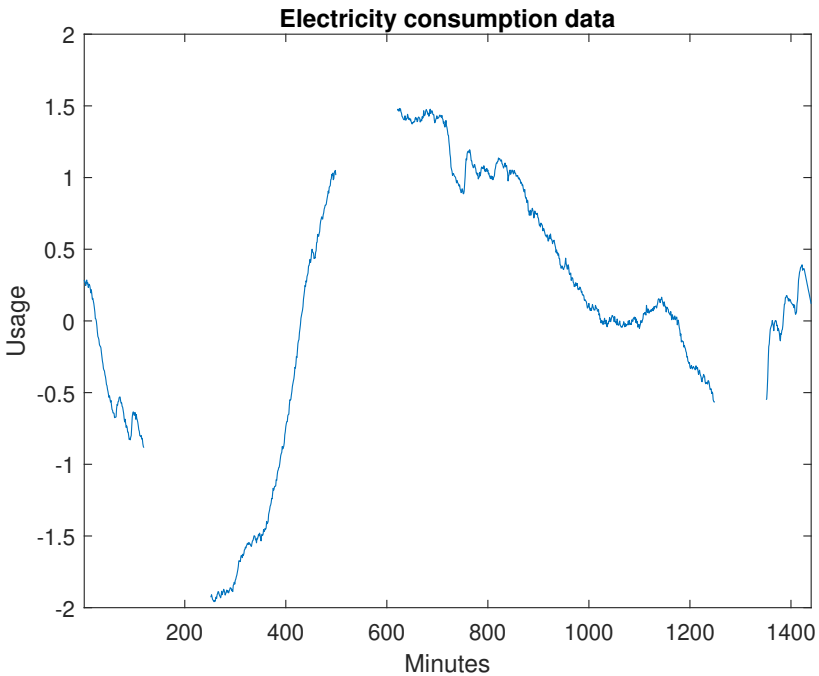
3. Detrending

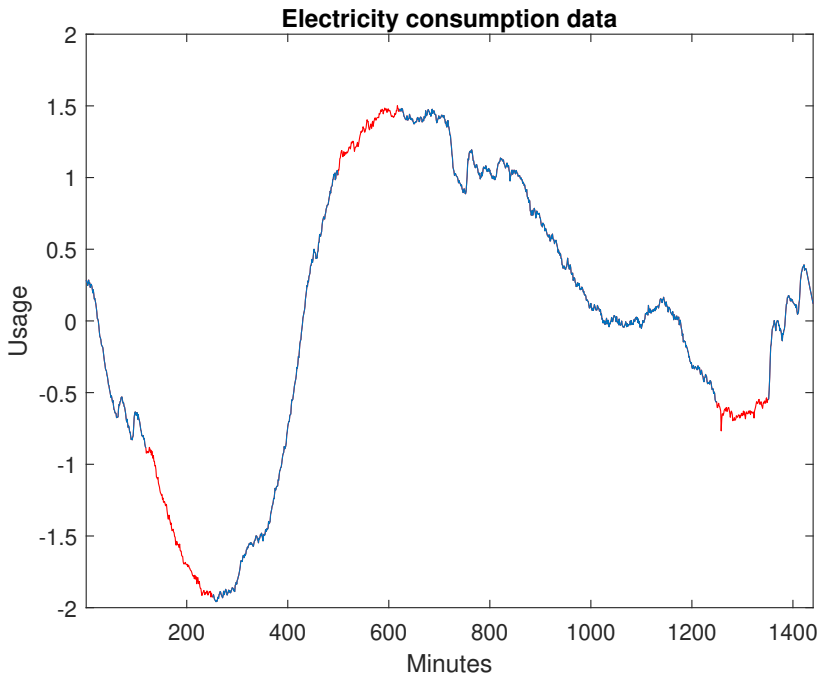
4. Interpolation of missing samples
4.1 Polynomial interpolation
4.2 Low-rank interpolation
4.3 Model-based interpolation

5. Outlier removal

6. Conclusion

Interpolation of missing samples





Interpolation of missing samples

Given a signal x and a set of missing samples $\mathcal{T}$, estimate the missing samples $\hat{\mathbf{x}}_{\mathcal{T}}$

Interpolation of missing samples

- **Missing data are very frequent :**
  - Sensor malfunctions
  - Clipping effect
  - Corrupted samples
- **Missing data can take several forms**
  - **Isolated samples**: easy to handle
  - **Contiguous samples** (up to 100): necessitates a full reconstruction
- Interpolation includes prediction and inpainting [Lepot et al., 2017]

Why are the samples missing?

Before choosing a method, one must ask why the samples are missing. Take a temperature sensor recording every hour [Rubin, 1976]

- **Missing Completely At Random (MCAR)** : the sensor randomly loses transmission packets. The probability of being missing depends on nothing
  - The observed samples are a representative sample of the complete series
  - The missing values can simply be ignored, or interpolated
- **Missing At Random (MAR)** : the sensor is solar-powered and fails more often at night. The probability depends on a variable that is observed (here, the hour)
  - More cold values are missing : ignoring them overestimates the mean
  - Unbiased provided the method conditions on the observed variable
- **Missing Not At Random (MNAR)** : the sensor saturates above 40°C and records nothing. The probability depends on the missing value itself
  - No observed variable can tell us what was lost
  - Any interpolation is biased, and will underestimate the extremes

Is it always useful to interpolate?

- Complex question that depends on the percentage of missing samples, their temporal positions with respect to each other, and the missingness generative process (cf previous slide)
- **Reconstructed samples are not data** : they should be flagged, and should not be counted in the sample size of a statistical test
- In order to use interpolated samples in the best possible way, one idea is to explicitly **model the missingness mechanism** itself in the global time series model (but it is very complex...)

Polynomial interpolation

Given a time series $\mathbf{x}$ that we want to interpolate on the integer set $\mathcal{T} = [\![n_{start}, n_{end}]\!]$, the easiest interpolation strategy consists in using polynomial models for the reconstruction

- **Constant value**

$$\forall n \in \mathcal{T}, \quad \hat{x}[n] = \frac{x[n_{start} - 1] + x[n_{end} + 1]}{2}$$

- **Linear interpolation**

$$\forall n \in \mathcal{T}, \quad \hat{x}[n] = \beta_1 n + \beta_0$$

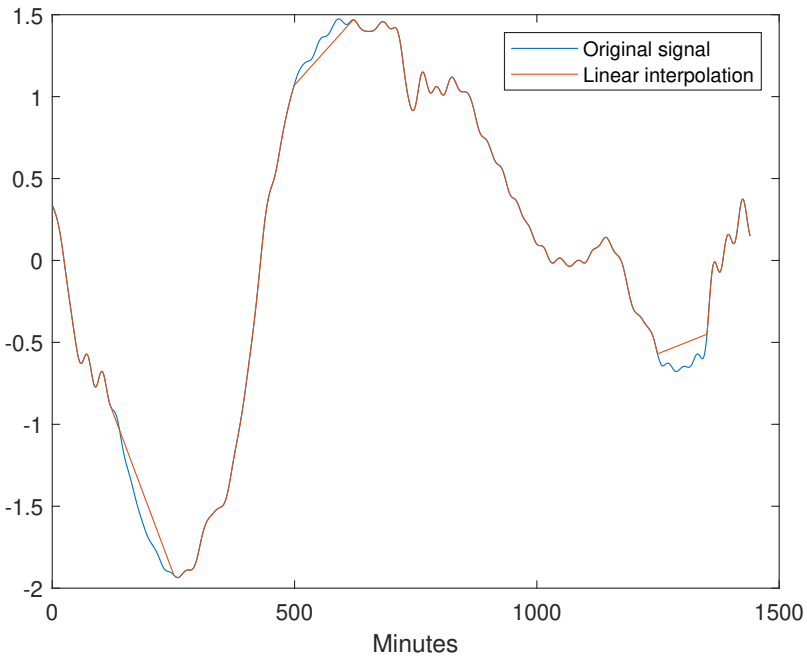
where β_0, β_1 are determined with the values $x[n_{start} - 1]$ and $x[n_{end} + 1]$

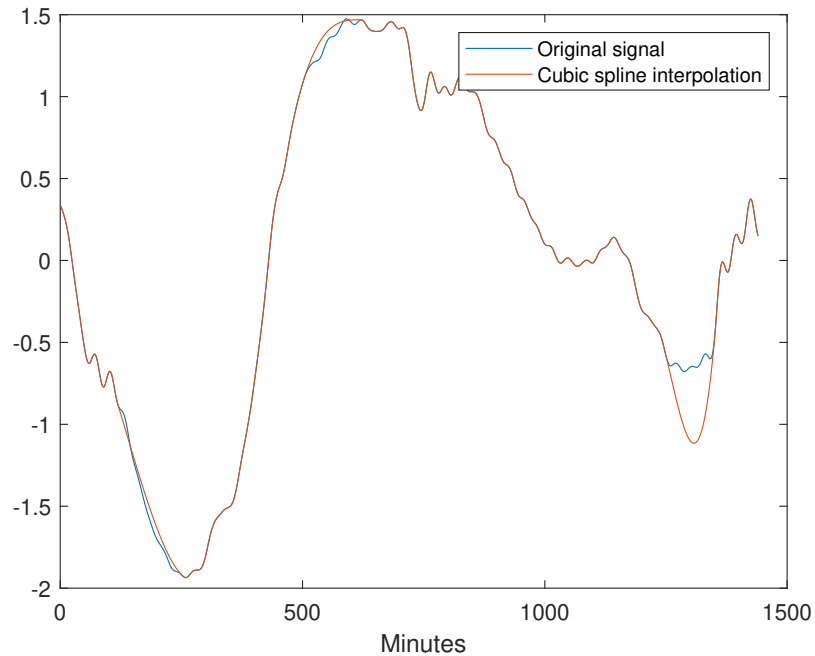
- **Cubic spline interpolation** [McKinley et al., 1998]

$$\forall n \in \mathcal{T}, \quad \hat{x}[n] = \beta_3 n^3 + \beta_2 n^2 + \beta_1 n + \beta_0$$

where β_k are determined by solving a system of equations based on $x[n_{start} - 2]$, $x[n_{start} - 1]$, $x[n_{end} + 1]$ and $x[n_{end} + 2]$

Example


Original signal
Linear interpolation
1.5
1
0.5
0
-0.5
-1
-1.5
-2
0
500
1000
1500
Minutes


Original signal
Cubic spline interpolation
1.5
1
0.5
0
-0.5
-1
-1.5
-2
0
500
1000
1500
Minutes

Pros and cons

- Easy to implement and good results for small segments
- In particular, **when only a few missing samples: constant or linear values are often the best**
- When the degree of the polynomial increases, **instabilities** may occur (strong dependency with the neighborhood samples)
- When used extensively, may lead to a smoothing of the signal hence a change in the spectrum (**boosting of the low frequencies**)

Low-rank interpolation

- The low-rank assumption on the trajectory matrix can also be used for reconstructing missing samples

$$\mathbf{X} = \begin{pmatrix} x[0] & \cdots & x[N-N_w] \\ x[1] & \cdots & x[N-N_w+1] \\ \vdots & \ddots & \vdots \\ x[N_w-1] & \cdots & x[N-1] \end{pmatrix}$$

- In this case, we will use the **Singular Value Decomposition** adapted to data with missing values [Srebro et al., 2003]
- These techniques are efficient for **medium-size missing patches**, as the low-rank assumption is usually only valid for relatively small windows

Principle

- The main idea is to compute a low-rank approximation of the trajectory matrix $\mathbf{X} \in \mathbb{R}^{N_w \times N_f}$, where only the **largest** singular values are kept

$$\hat{\mathbf{X}} = \sum_{k=1}^{K} \lambda_k \mathbf{u}_k \mathbf{v}_k^t$$

  where $K < \min(N_w, N_f)$
- But how do we compute the SVD for a matrix that contains missing values?
- **Mask matrix:**

$$W_{i,j} = \begin{cases} 0 & \text{if } X_{i,j} \text{ is missing} \\ 1 & \text{else} \end{cases}$$

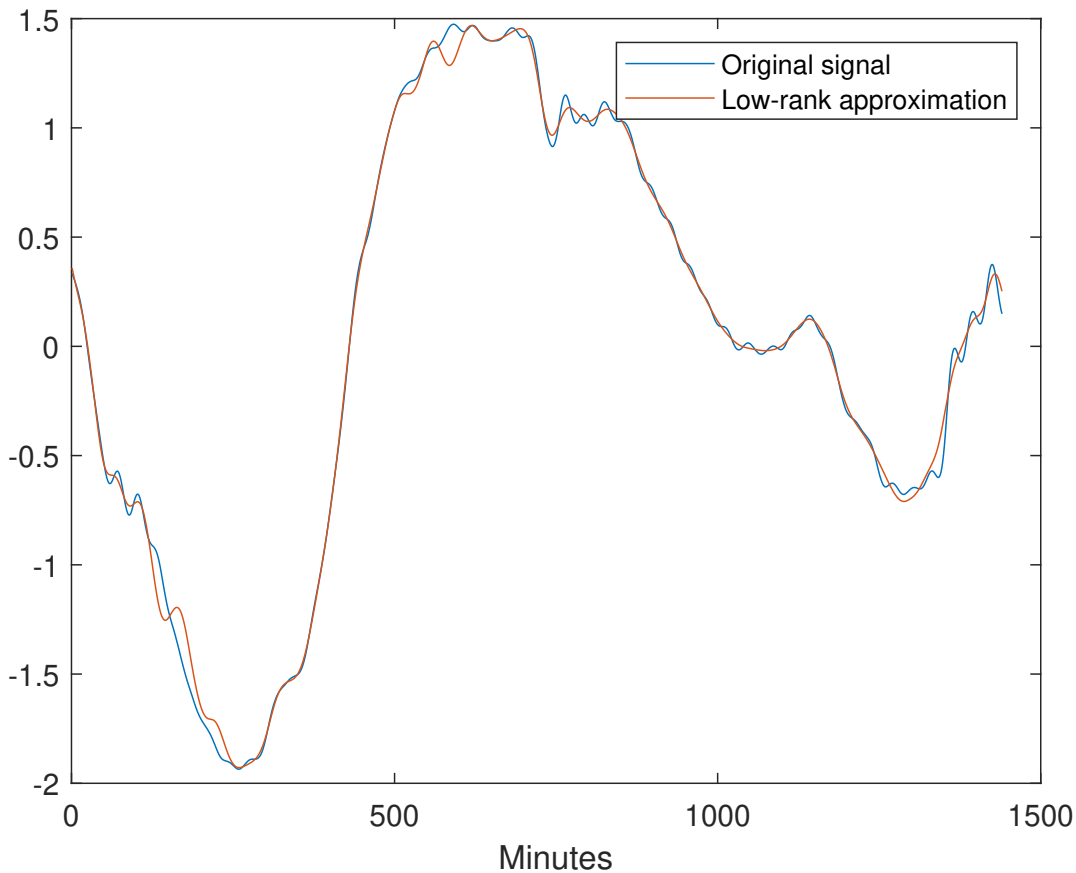
- Low-rank approximation will only be used to update the missing samples

Low-rank interpolation

Algorithm 1: Low-rank interpolation

Input : Trajectory matrix $\mathbf{X}$ with missing values
Mask matrix $\mathbf{W}$
Expected rank K
Output: Interpolated trajectory matrix $\hat{\mathbf{X}}$
Initialize $\hat{\mathbf{X}}$;
while $n_{iter} < n_{max}$ **do**
  SVD computation;
  $[\mathbf{U}, \mathbf{\Lambda}, \mathbf{V}] = \text{SVD}\left(\mathbf{X} \odot \mathbf{W} + \hat{\mathbf{X}} \odot (\mathbf{1} - \mathbf{W})\right)$;
  Low-rank approximation;
  $\hat{\mathbf{X}} = \sum_{k=1}^{K} \lambda_k \mathbf{u}_k \mathbf{v}_k^t$
end

Example



$$N_w = 300,\ K = 10$$

Pros and cons

- Good results for medium size segments
- Compromise: always set N_w **greater than the largest missing patch**, but when N_w becomes too large, the low-rank approximation becomes less valid
- Variant with adaptive rank can provide better results [Srebro et al., 2003]

$$K_{n_{iter}} = \max(\min(N_w, N_f) - n_{iter}, K)$$

- Rank K can be estimated by computing the SVD of the initialized trajectory matrix (with polynomial reconstruction for instance)

Model-based interpolation

- For long segments of missing samples, interpolation becomes a **full reconstruction** task
- In this case a model is necessary to obtain a satisfactory interpolation
  1. Choice of an adequate model
  2. Parameter inference from the known samples
  3. Replacement of the missing samples by values in adequacy with the learned model

Model-based interpolation

- **Problem**: how do we estimate the parameters from a time series with missing data?
- Iterative solution
  1. **Initialization** of the missing samples with simple rough estimates (set to zero, constant or linear interpolation...)
  2. **Parameter inference** from all samples
  3. **Reconstruction** of the missing samples from the learned model
  4. Repeat steps 2 and 3 until convergence

AR-based interpolation

- For an $AR(p)$ model, given estimates of parameters $\hat{\mathbf{a}}$, the signal can be reconstructed by assuming that

$$x[n] \approx -\sum_{i=1}^{p} \hat{a}_i x[n-i]$$

- The prediction error on the whole time series writes

$$E(\mathbf{x}) = \sum_{n=p}^{N-1} \left| x[n] + \sum_{i=1}^{p} \hat{a}_i x[n-i] \right|^2$$

- The main idea is to minimize this quantity in order to retrieve appropriate values for the missing samples [Janssen et al., 1986]

AR-based interpolation

$$\mathbf{x}^* = \underset{\tilde{\mathbf{x}} : \forall n \notin \mathcal{T},\ \tilde{x}[n]=x[n]}{\operatorname{argmin}} E(\tilde{\mathbf{x}})$$

- This optimization problem has a **closed form solution** (least-squares estimates) that is obtained by rewriting $E(\mathbf{x})$ as the sum of terms depending on the missing samples $n \in \mathcal{T}$ and other depending only on the known samples.
- By denoting $\mathbf{x}_{\mathcal{T}}$ the set of missing samples, the equation rewrites

$$E(\mathbf{x}) = {\mathbf{x}_{\mathcal{T}}}^T \mathbf{B} \mathbf{x}_{\mathcal{T}} + 2{\mathbf{x}_{\mathcal{T}}}^T \mathbf{d} + C$$

where

  - $\forall (t, t') \in \mathcal{T},\ \ b_{t,t'} = \begin{cases} \sum_{l=0}^{p-|t-t'|} \hat{a}_l \hat{a}_{l+|t-t'|} & \text{if } 0 \leq |t-t'| \leq p \\ 0 & \text{else} \end{cases}$
  - $\forall t \in \mathcal{T},\ \ d_t = \sum_{\substack{-p \leq k \leq p \\ t-k \notin \mathcal{T}}} b_{|k|} x[t-k]$
  - C is a constant only depending on the known samples
- The final problem is simply a **linear system** and thus easy to solve

$$\mathbf{B} \mathbf{x}_{\mathcal{T}} = -\mathbf{d}$$

AR-based interpolation

Algorithm 2: AR-based interpolation

Inputs : Time series $\mathbf{x} \in \mathbb{R}^N$ with missing values
Set of missing samples $\mathcal{T}$
AR model order p

Output: Interpolated samples $\hat{\mathbf{x}}_{\mathcal{T}}$

$\hat{\mathbf{x}}_{\mathcal{T}} = \mathbf{0}_{|\mathcal{T}|}$;

while $n_{iter} < n_{max}$ **do**

  AR estimation step;

  Estimate $\hat{\mathbf{a}}$ with the Levinson-Durbin algorithm;

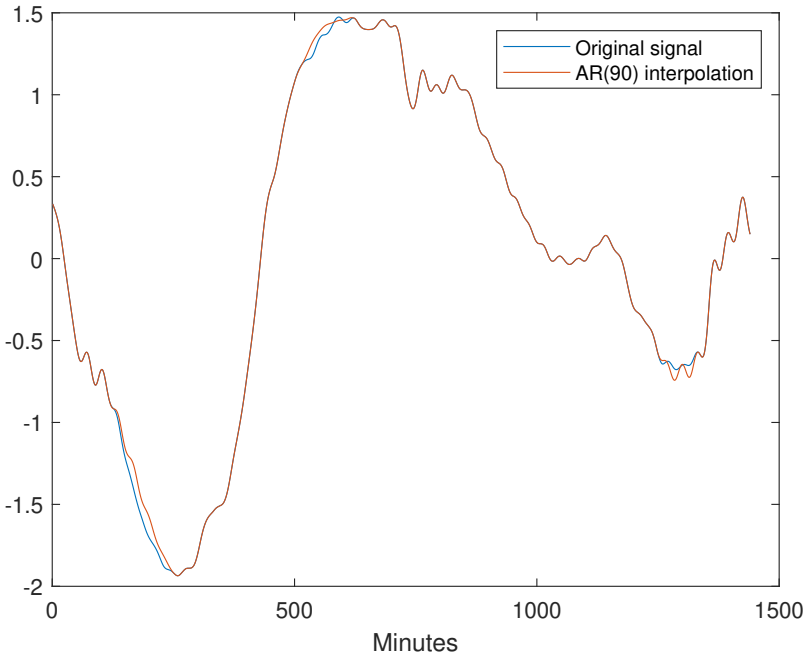
  AR interpolation step;

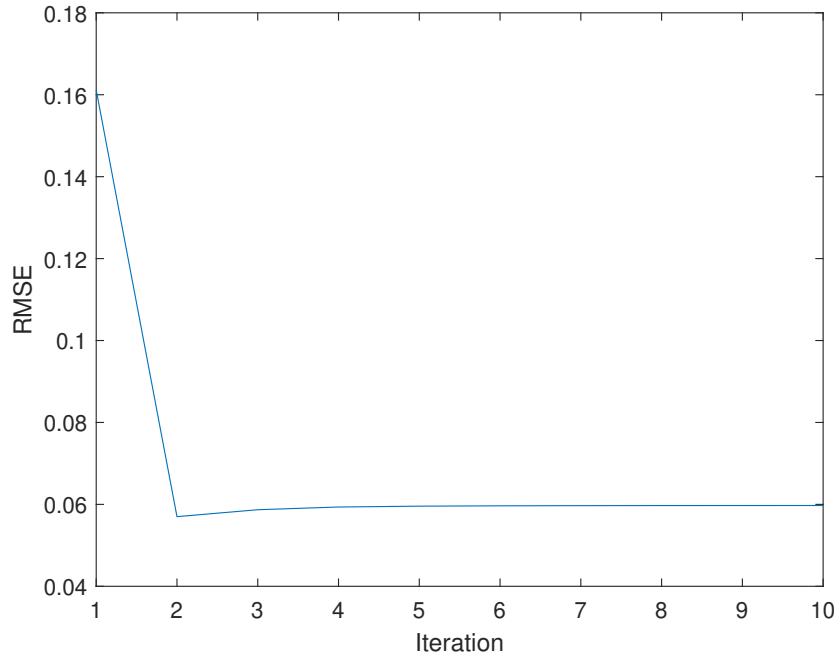
  Compute $\mathbf{B}$ and $\mathbf{d}$ and solve for $\hat{\mathbf{x}}_{\mathcal{T}}$;

  Set $\mathbf{x}_{\mathcal{T}} = \hat{\mathbf{x}}_{\mathcal{T}}$;

end

Example





Interpolation with $AR(90)$ model

Contents

1. Problem statement

2. Denoising

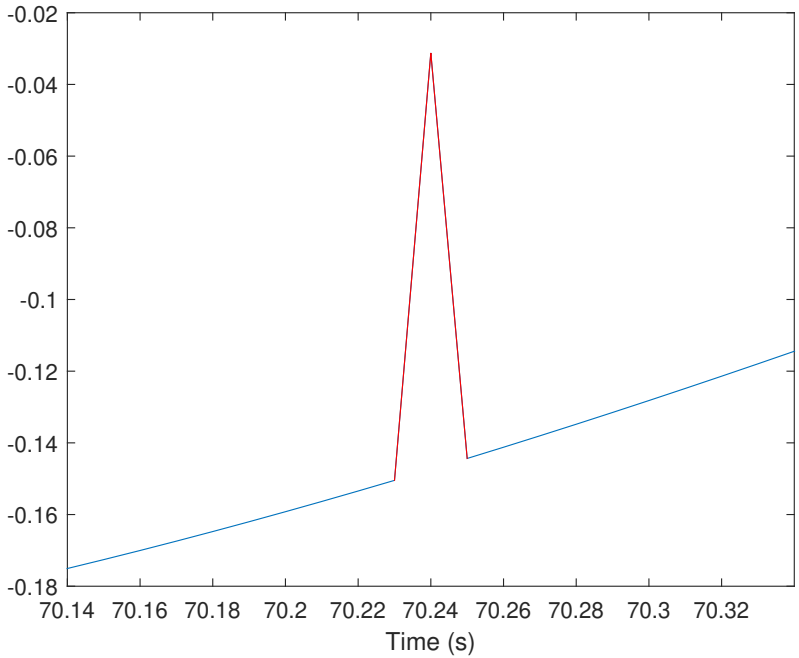
3. Detrending

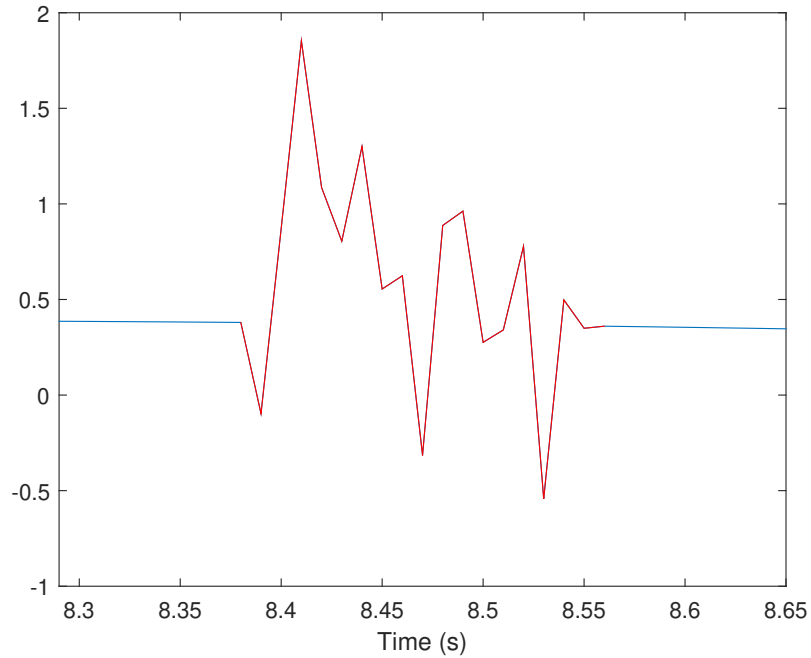
4. Interpolation of missing samples

5. Outlier removal
5.1 Isolated samples
5.2 Contiguous samples

6. Conclusion

Outlier removal





Outliers, also called impulsive noise (as opposed to AWGN) correspond to spurious samples (isolated or continuous) that take unlikely values

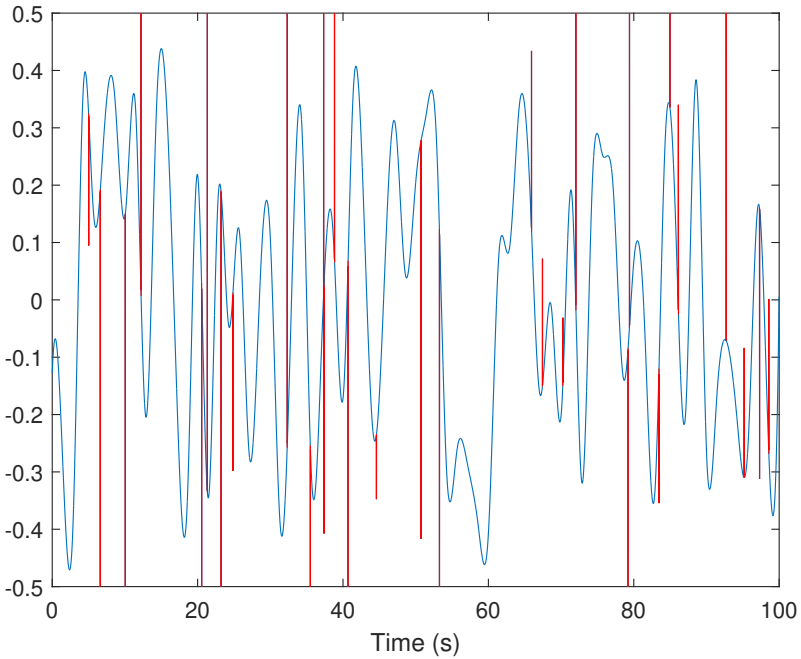
Outlier removal

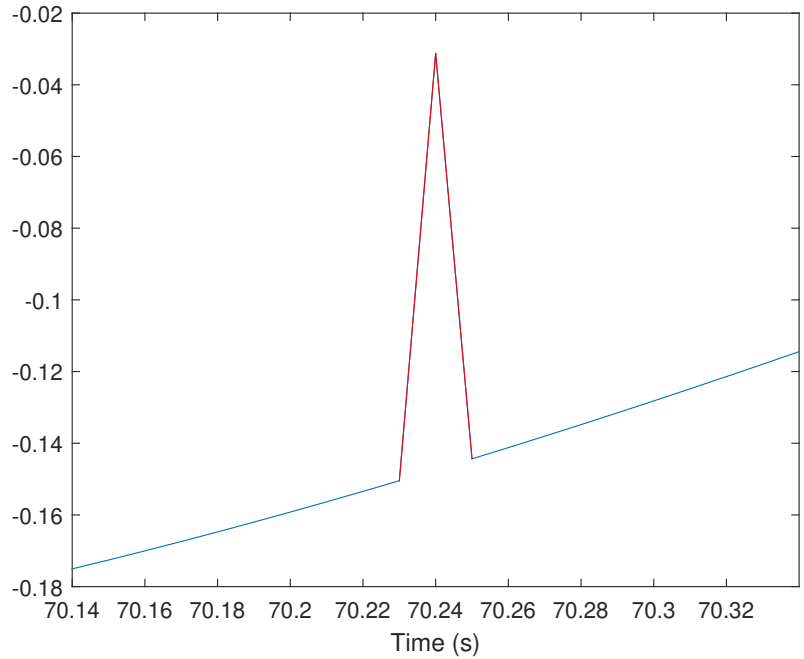
Outlier removal

Given a signal $x[n]$, outlier removal consists in detecting the locations $\mathcal{T}$ of the outliers (detection phase) and to replace these values with more adequate values (interpolation phase)

- Interpolation phase can be done by using the previously described algorithms. We will therefore focus on the **detection phase.**
- Two settings: isolated samples or contiguous group of samples
- Outliers are not only characterized by their values but also on their positions in the time series: **context is fundamental**

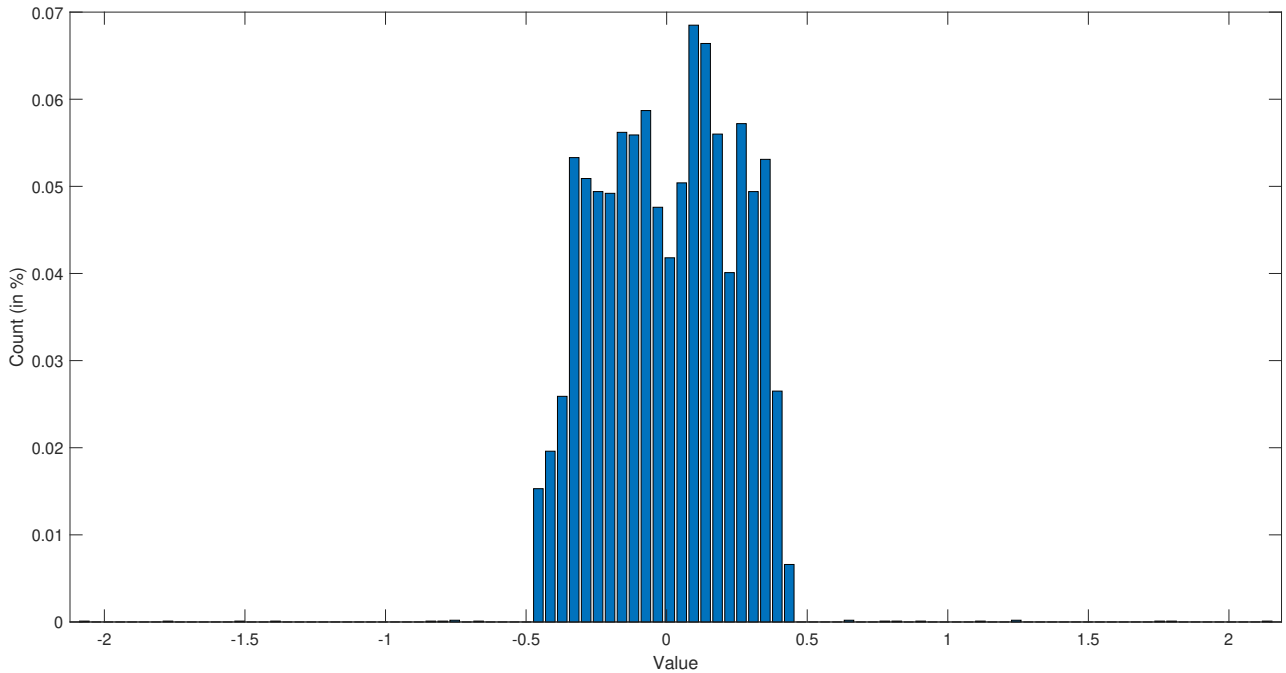
Isolated samples





Impulsive noise that only corrupts isolated samples

Histogram



If the values taken by the impulsive noise are particularly large with respect to the signal, they can be detected by looking at the histogram of the values taken by the samples: similar to outlier detection in statistical data

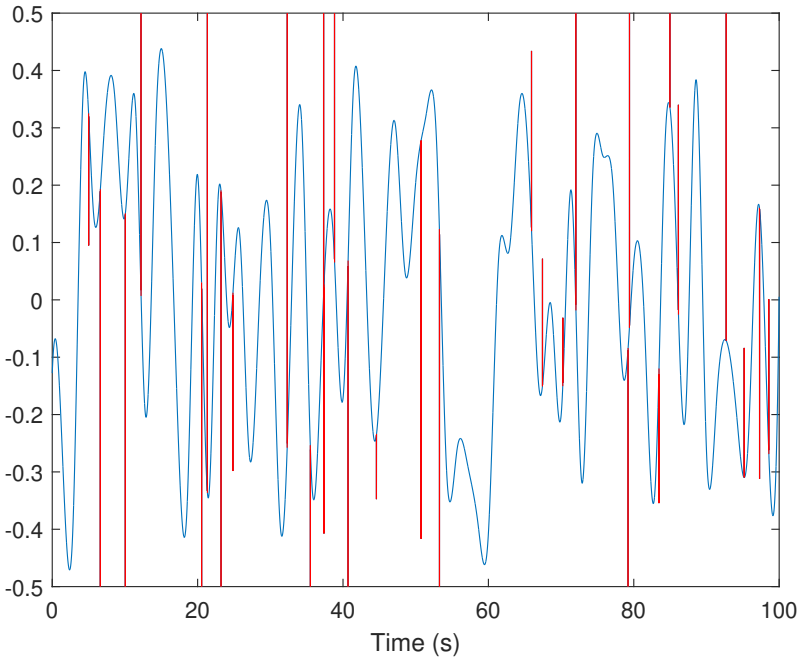
Median filtering

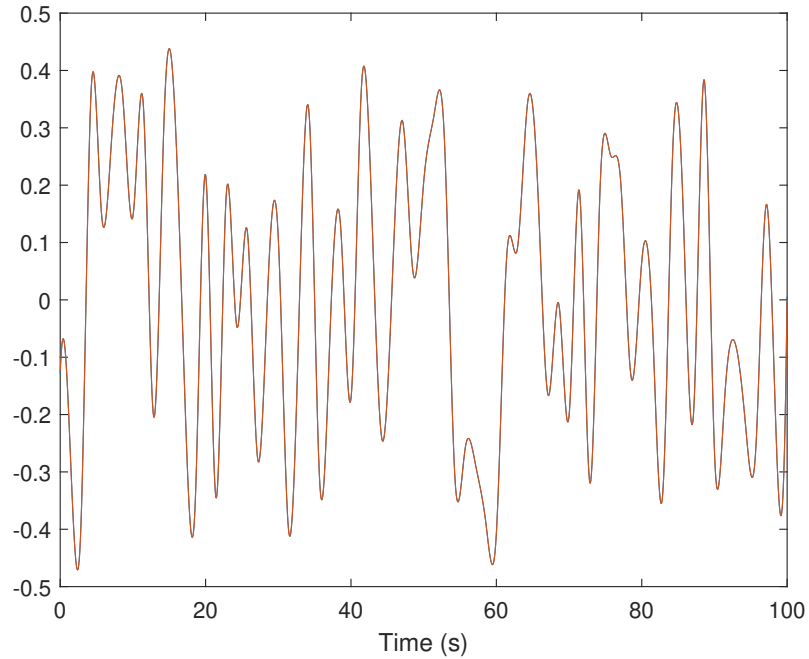
- Outliers can be detected AND removed by using a sliding median filtering that replaces each value by the median of the samples in a window of length $2w+1$:

$$\hat{x}[n] = \text{median}_{-w \leq i \leq +w} \{x[n-i]\}$$

- Median filtering **smooths** the time series while preserving the discontinuities
- Example : original signal $[0.3 \;\; 0.4 \;\; 0.45]$ and noisy signal $[0.3 \;\; 9 \;\; 0.45]$
  - Moving average filter: $0.9 \rightarrow 3.25$
  - Median filter: $0.9 \rightarrow 0.45$

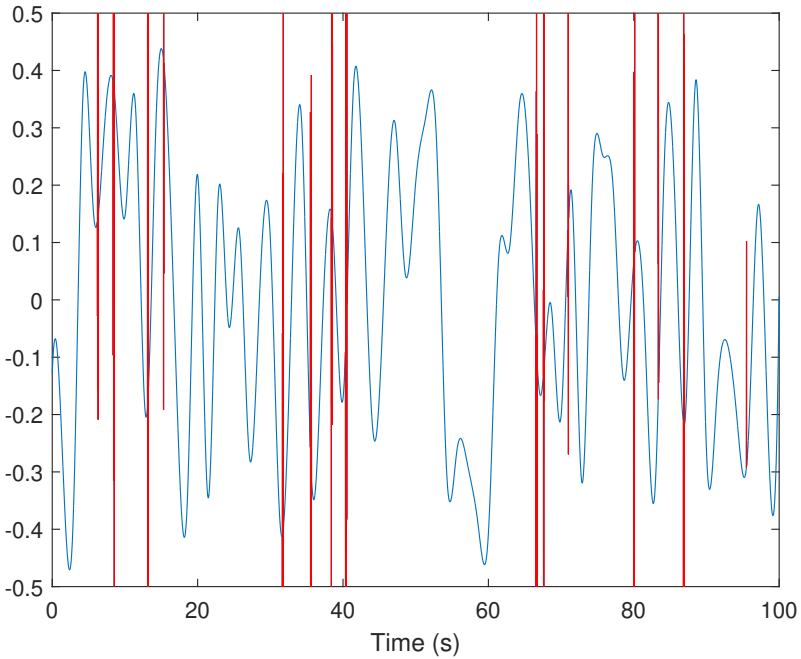
Median filtering

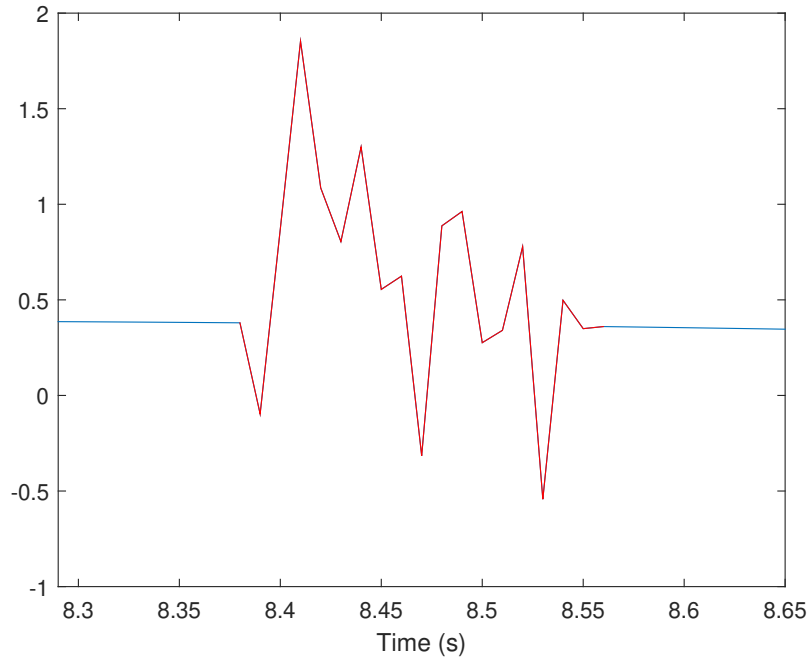




Perfect reconstruction with median filtering ($2w + 1 = 3$ samples)

Contiguous samples





Impulsive noise that corrupts groups of contiguous samples

Contiguous samples

- When the impulsive noise corrupts groups of contiguous samples, studying the values is **not sufficient**
- In order to retrieve the set of outliers $\mathcal{T}$, using a model may be necessary
- Outliers: samples that are **far** from their predicted values according to a model
- Same principle as model-based interpolation: parameter estimation, detection, interpolation and reiterate
- Note: this task is close to the Anomaly Detection task (see Lecture 5)

AR-based outlier detection

$$x[n] = -\sum_{i=1}^{p} a_i x[n-i] + b[n]$$

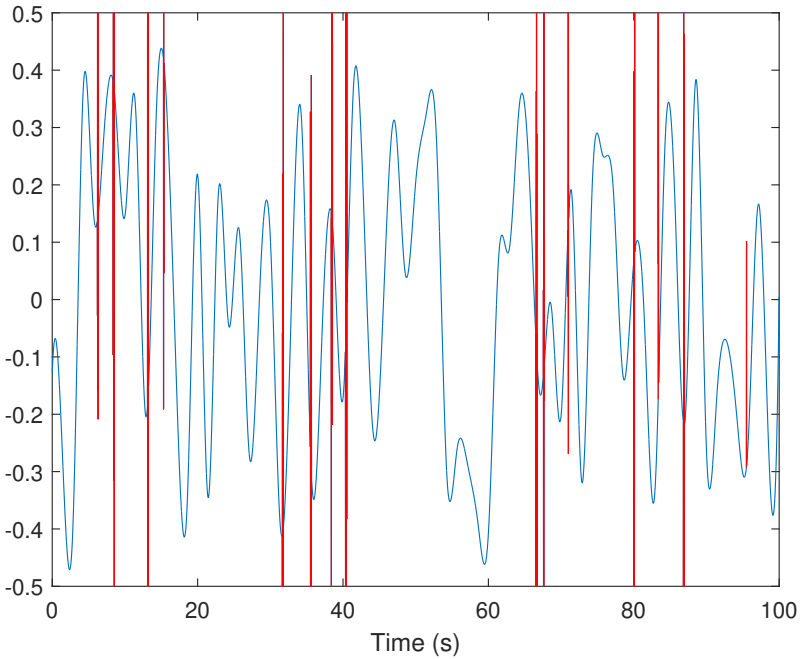
- Given estimates of the AR parameters $\hat{\mathbf{a}}$, the prediction error writes:

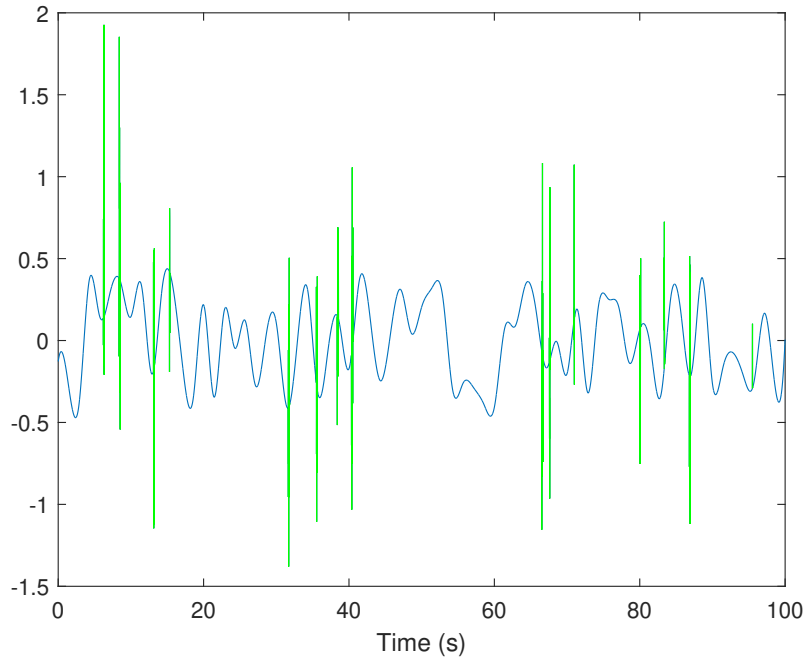
$$e[n] = x[n] + \sum_{i=1}^{p} \hat{a}_i x[n-i]$$

- If adapted model, good parameter estimation and low noise variance, this quantity must be rather small for samples that are not outliers [Jenssen, 1986]
- **Detection method** with threshold λ :

$$\mathcal{T} = \{n \text{ s.t. } |e[n]| > \lambda\}$$

AR-based outlier detection





Detection with $AR(10)$ model

AR-based outlier detection and removal

In order to perform both detection and removal of impulsive noise, alternance between

1. **Estimation step**: learn the AR parameters from the current time series
2. **Detection step**: detect the set of outliers
3. **Interpolation step**: replace these outliers by appropriate values
4. Reiterate steps 1, 2, 3

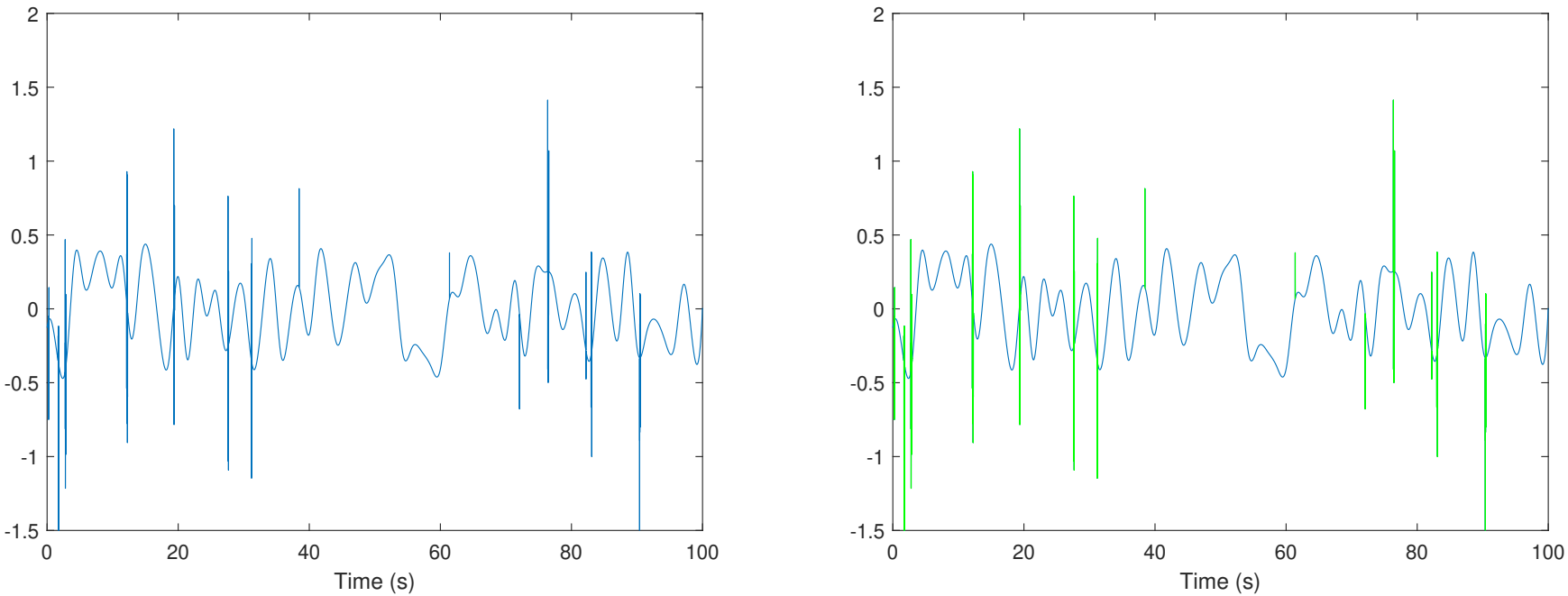
AR-based outlier detection and removal

Algorithm 3: AR-based outlier detection and removal

Inputs : Time series $\mathbf{x} \in \mathbb{R}^N$ with outliers
AR model order p, Threshold λ
Output: Denoised time series $\hat{\mathbf{x}} \in \mathbb{R}^N$

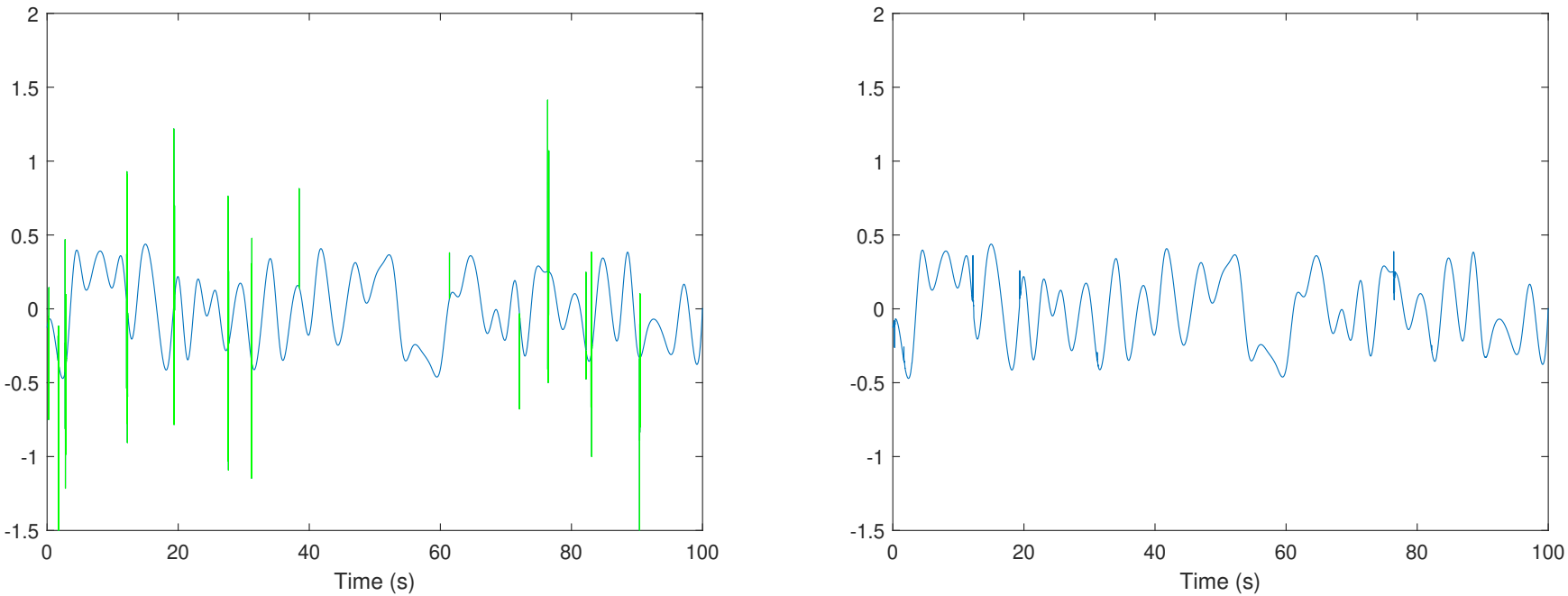
$\hat{\mathbf{x}} = \mathbf{x}$;
while $n_{iter} < n_{max}$ **do**
    AR estimation step;
    Estimate $\hat{\mathbf{a}}$ from $\hat{\mathbf{x}}$ with the Levinson-Durbin algorithm;
    Detection step;
    Compute $\mathbf{e}$ and set $\mathcal{T} = \{n \text{ s.t. } |e[n]| > \lambda\}$;
    AR interpolation step;
    Compute $\mathbf{B}$ and $\mathbf{d}$ and solve for $\hat{\mathbf{x}}_{\mathcal{T}}$;
end

Example



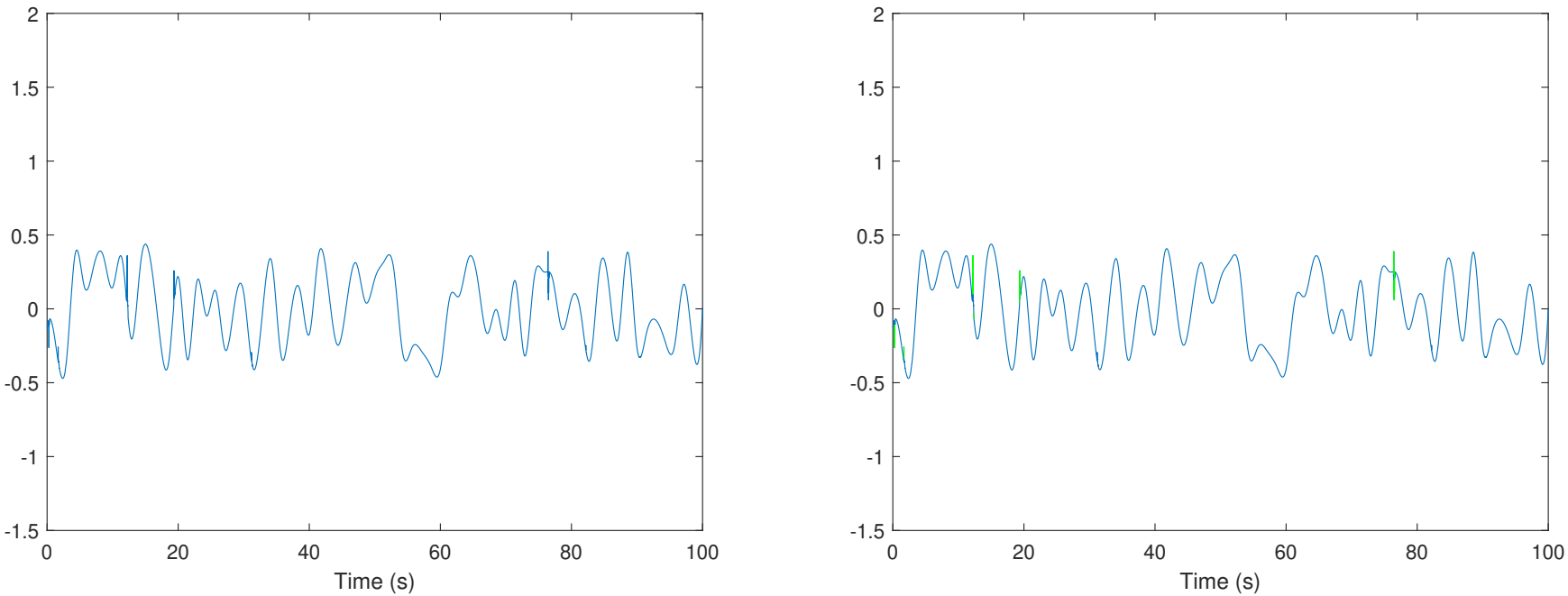
Iteration 1: Detection with $AR(10)$ model

Example



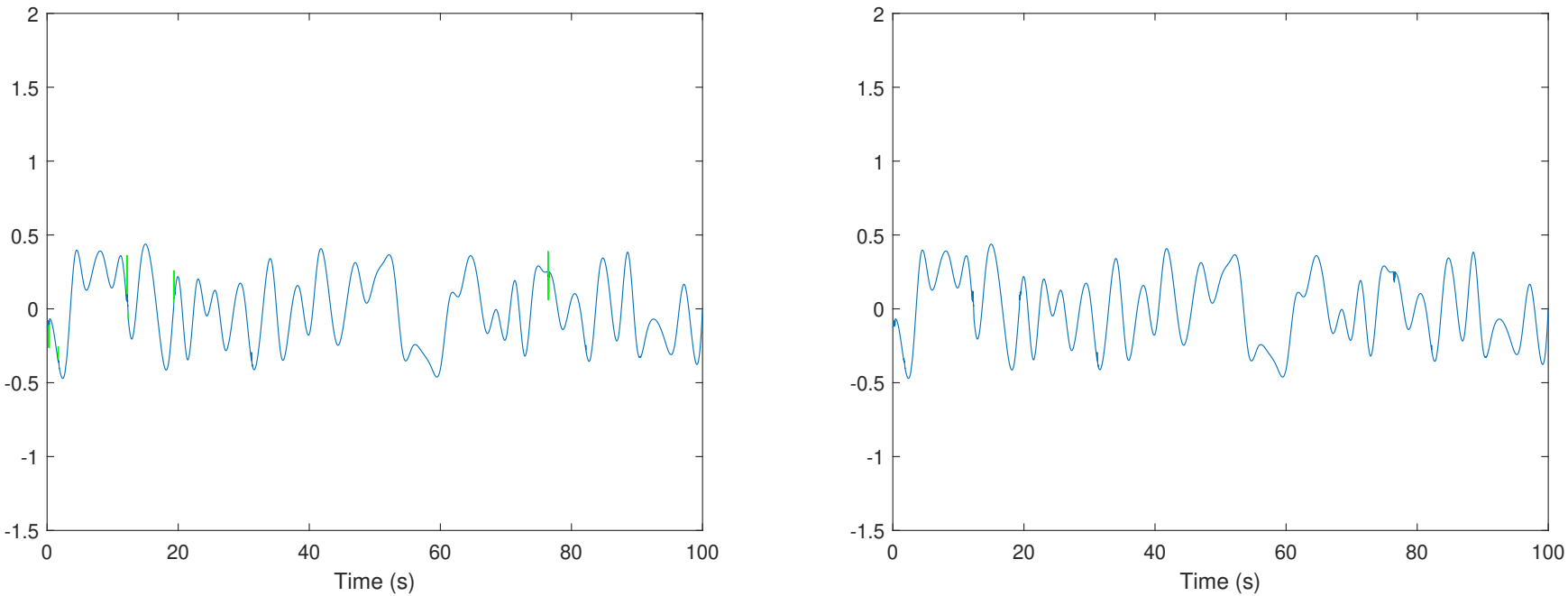
Iteration 1: Interpolation with $AR(10)$ model

Example



Iteration 2: Detection with $AR(10)$ model

Example



Iteration 2: Interpolation with $AR(10)$ model

Contents

1. Problem statement

2. Denoising

3. Detrending

4. Interpolation of missing samples

5. Outlier removal

6. Conclusion

How to apply preprocessings

- In this lecture, all preprocessing steps have been discussed separately. However, it is likely that a single time series may be affected by **multiple perturbations**
- **These perturbations can interact with one another**, thereby creating complicated situations
  Some examples:
  - The presence of outliers can disrupt all preprocessing methods based on model estimates
  - The presence of a trend will disrupt methods based on spectral estimation
  - In the case of missing samples or irregular sampling rates, it is impossible to use Fourier-based techniques
- It is therefore important to apply the successive preprocessing steps in a logical order
- In most cases it is advised to do outliers, then missing samples, then trend, then noise.
- But the most important is to check and to keep track of all these steps, as discussed previously!

How to evaluate preprocessing

- Preprocessing often constitutes only a small part of a larger processing pipeline, and it is rare for it to be properly evaluated
- One way to evaluate the preprocessings is to use **synthetic ground truth** signals : corrupt a clean signal in a controlled way and measure the reconstruction error
- For some time series it is also possible to isolate "silent parts" where only perturbations (no signal) occur
- In most cases, preprocessings are assessed by running the whole pipeline and measuring the final performances
- Always use a validation set to learn the preprocessing parameters: estimating them by using the final scores is wrong

Conclusion

- **Preprocessings is crucial**
  - Tedious task, but one that plays a key role in improving (or worsening) performance
  - Information on preprocessing is missing from most of published work which raises serious issues regarding reproducibility
- **Preprocessing is almost never evaluated**
  - It is performed before any model is fitted, and is therefore not part of the validation applied to the model
  - There is no benchmark, because there is no ground truth on real recordings
- **A neglected field seen as "has been"**
  - Denoising and detrending have almost disappeared from the recent literature : the tools are considered settled
  - Imputation is the exception, but it has become a **task in its own right**, with deep architectures and benchmarks, rather than a preprocessing step

A few reference papers

- Mallat, S. (2008). A Wavelet Tour of Signal Processing: The Sparse Way (3rd edition). Academic Press. : book on sparse representations
- Donoho, D. L., & Johnstone, I. M. (1994). Ideal spatial adaptation by wavelet shrinkage. Biometrika, 81(3), 425-455. : thresholding rules and their theoretical guarantees
- Golyandina, N., & Zhigljavsky, A. (2013). Singular Spectrum Analysis for Time Series. Springer. : book on singular spectrum analysis
- Cleveland, R. B., Cleveland, W. S., McRae, J. E., & Terpenning, I. (1990). STL: a seasonal-trend decomposition procedure based on loess. Journal of Official Statistics, 6(1), 3-73. : seasonal-trend decomposition
- Little, R. J. A., & Rubin, D. B. (2019). Statistical Analysis with Missing Data (3rd edition). Wiley. : book on missing data mechanisms and imputation
- Lepot, M., Aubin, J. B., & Clemens, F. H. (2017). Interpolation in time series: an introductive overview of existing methods, their performance criteria and uncertainty assessment. Water, 9(10), 796. : survey on interpolation methods and their evaluation

Code and resources

- `scipy.signal` - https://docs.scipy.org/doc/scipy/reference/signal.html
  Filter design, median filtering, detrending
- `PyWavelets` - https://pywavelets.readthedocs.io
  Wavelet transforms and thresholding
- `statsmodels` - https://www.statsmodels.org
  Seasonal decomposition, STL, autoregressive models with missing observations
- `scikit-learn` - https://scikit-learn.org
  Dictionary learning, orthogonal matching pursuit, truncated singular value decomposition
- `PyEMD` - https://pyemd.readthedocs.io
  Empirical mode decomposition
- `PyPOTS` - https://pypots.com
  Methods for partially observed time series, with a benchmark of imputation algorithms

References

- Rubinstein, R., Bruckstein, A. M., & Elad, M. (2010). Dictionaries for sparse representation modeling. Proceedings of the IEEE, 98(6), 1045-1057.
- Percival, D. B., & Walden, A. T. (2000). Wavelet methods for time series analysis (Vol. 4). Cambridge university press.
- Mallat, S. (1999). A wavelet tour of signal processing. Elsevier.
- Vautard, R., Yiou, P., & Ghil, M. (1992). Singular-spectrum analysis: A toolkit for short, noisy chaotic signals. Physica D: Nonlinear Phenomena, 58(1-4), 95-126.
- Flandrin, P., Goncalves, P., & Rilling, G. (2004, September). Detrending and denoising with empirical mode decompositions. In 2004 12th European Signal Processing Conference (pp. 1581-1584). IEEE.
- Boudraa, A. O., & Cexus, J. C. (2006). Denoising via empirical mode decomposition. Proc. IEEE ISCCSP, 4(2006).
- Comon, P. (1994). Independent component analysis, a new concept?. Signal processing, 36(3), 287-314.
- Lepot, M., Aubin, J. B., & Clemens, F. H. (2017). Interpolation in time series: An introductive overview of existing methods, their performance criteria and uncertainty assessment. Water, 9(10), 796.
- McKinley, S., & Levine, M. (1998). Cubic spline interpolation. College of the Redwoods, 45(1), 1049-1060.
- Srebro, N., & Jaakkola, T. (2003). Weighted low-rank approximations. In Proceedings of the 20th International Conference on Machine Learning (ICML-03) (pp. 720-727).
- Janssen, A. J. E. M., Veldhuis, R., & Vries, L. (1986). Adaptive interpolation of discrete-time signals that can be modeled as autoregressive processes. IEEE Transactions on Acoustics, Speech, and Signal Processing, 34(2), 317-330.
- Oudre, L. (2015). Automatic detection and removal of impulsive noise in audio signals. Image Processing On Line, 5, 267-281.

Lecture 4: list of possible topics/projects (1/2)

Denoising

Project 4.1 Bayer, F. M., Kozakevicius, A. J., & Cintra, R. J. (2019). An iterative wavelet threshold for signal denoising. Signal Processing, 162, 10-20.
How to use adaptive wavelet thresholding for denoising

Project 4.2 Moussallam, M., Gramfort, A., Daudet, L., & Richard, G. (2014). Blind denoising with random greedy pursuits. IEEE Signal Processing Letters, 21(11), 1341-1345.
How to use statistical considerations to set the parameters in greedy denoising approaches

Project 4.3 Gavish, M., & Donoho, D. L. (2014). The optimal hard threshold for singular values is $4/\sqrt{3}$. IEEE Transactions on Information Theory, 60(8), 5040-5053.
How to choose the number of singular values to keep, instead of reading an elbow

Detrending and decomposition

Project 4.4 Flandrin, P., Gonçalves, P., & Rilling, G. (2004). Detrending and denoising with empirical mode decompositions. In European Signal Processing Conference (pp. 1581-1584).
How to use empirical mode decomposition for denoising and detrending

Project 4.5 Wen, Q., Gao, J., Song, X., Sun, L., Xu, H., & Zhu, S. (2019). RobustSTL: a robust seasonal-trend decomposition algorithm for long time series. In Proceedings of the AAAI Conference on Artificial Intelligence (pp. 5409-5416).
How to extract trend and seasonality jointly when the signal contains abrupt changes

Project 4.6 Harmouche, J., Fourer, D., Auger, F., Borgnat, P., & Flandrin, P. (2018). The sliding singular spectrum analysis: a data-driven nonstationary signal decomposition tool. IEEE Transactions on Signal Processing, 66(1), 251-263.
How to extend singular spectrum analysis to signals whose content varies over time

Lecture 4: list of possible topics/projects (2/2)

Interpolation of missing samples

Project 4.7 Adler, A., Emiya, V., Jafari, M. G., Elad, M., Gribonval, R., & Plumbley, M. D. (2012). Audio inpainting. IEEE Transactions on Audio, Speech, and Language Processing, 20(3), 922-932.
How to use sparse representations to reconstruct missing segments

Project 4.8 Che, Z., Purushotham, S., Cho, K., Sontag, D., & Liu, Y. (2018). Recurrent neural networks for multivariate time series with missing values. Scientific Reports, 8(1), 6085.
How to handle missing values inside the model, instead of imputing them beforehand

Project 4.9 Du, W., Côté, D., & Liu, Y. (2023). SAITS: self-attention-based imputation for time series. Expert Systems with Applications, 219, 119619.
How to impute multivariate time series with an attention-based model

Project 4.10 Khayati, M., Lerner, A., Tymchenko, Z., & Cudré-Mauroux, P. (2020). Mind the gap: an experimental evaluation of imputation of missing values techniques in time series. Proceedings of the VLDB Endowment, 13(5), 768-782.
How to compare imputation methods fairly, and on which criteria

Outlier removal

Project 4.11 Pearson, R. K., Neuvo, Y., Astola, J., & Gabbouj, M. (2016). Generalized Hampel filters. EURASIP Journal on Advances in Signal Processing, 2016(1), 87.
How to detect and replace isolated outliers using robust local statistics

Project 4.12 Mokrý, O., & Rajmic, P. (2024). Tweaking autoregressive methods for inpainting of gaps in audio signals. arXiv:2403.04433.
How to improve autoregressive reconstruction of corrupted segments